

Design and Evaluation of a Deterministic Query Orchestrator for a Distributed Hybrid Retrieval System

Msc Computer Science



**UNIVERSITY
OF LONDON**

Software Project Report
Devang Chauhan

10402 Words

Excluding title page, abstract, table of content, tables, references and appendix

Abstract

Placeholder

Table of Contents

Abstract	2
Table of Contents	3
List of Figures	7
List of Tables.....	8
1. Introduction.....	9
1.1 Problem Statement.....	9
1.2 Contribution	10
1.3 Aim and Objectives.....	10
2. Literature Review	12
2.1 RAG.....	12
2.2 NL2SQL.....	12
2.3 Resource Selection	13
2.4 Vector Search.....	13
2.5 Relational Search	14
2.6 Distributed Systems	14
2.7 Query Planning.....	14
2.8 Synthesis and Research Gap.....	14
3. Methodology	16
3.1 Scope	16
3.2 Database	17
3.2.1 Selection and Preparation	18
3.2.2 Layout and Sharding	19
3.2.4 Embeddings	22

3.2.5 Indexes	23
3.3 Software Architecture	23
3.3.1 Orchestrator	24
3.3.2 Relational Orchestration	24
3.3.3 Vector Retrieval and Enrichment	27
3.3.4 Validation and Timing	30
3.4 Verification and Testing	30
3.5 Evaluation Method	30
3.5.1 Benchmark Construction	30
3.5.2 Configuration Coverage	31
3.5.3 Measurements	31
3.5.4 Execution Procedure	32
3.5.5 Bootstrapping	33
3.6 Reproducibility	37
3.7 Data Use and Credentials	37
4. Results and Analysis	38
4.1 Benchmark Outcomes	38
4.2 Verification Findings	40
4.3 Relational Retrieval and Shard Routing	41
4.4 Filter Order and Vector Latency	41
4.5 Retrieval Effectiveness and Result Completeness	43
4.6 Layout, Routing and Timing Boundaries	45
5. Evaluation	47
5.1 Achievement of the Objectives	47
5.2 Interpretation in Relation to the Literature Review	47
5.3 Validity and Generalisability	48

5.3.1 Workload and Relevance Assessment	48
5.3.2 Experimental Control and Statistical Interpretation	49
5.3.3 Deployment and Configuration Boundaries.....	49
5.4 Engineering Strengths and Remaining Limitations	50
5.5 Further Work.....	50
6. Conclusion.....	52
References	53
Appendix A	58
A.1 HTTP Endpoints.....	58
A.2 Errors	58
Appendix B	60
B.1 Sample Queries and Responses	60
Appendix C.....	63
C.1 Test Requirements to Implementation Mapping Summary	63
C.1.1 Execute valid requests.....	63
C.1.2 Follow caller-supplied routing	63
C.1.3 Reject invalid inputs	64
C.1.4 Validate original requests.....	64
C.1.5 Merge and limit ranked results.....	64
C.1.6 Preserve ranking during enrichment.....	65
C.1.7 Apply filter semantics consistently	65
C.1.8 Use canonical industry labels	66
C.1.9 Report dependency failures	66
C.1.10 Enforce read-only execution	67
C.1.11 Validate benchmark procedure	67
C.1.12 Detect missing relational records.....	67

Appendix D	69
------------------	----

List of Figures

Figure 3.1 Project scope	17
Figure 3.2 Logical shards	18
Figure 3.3 Industry classification	19
Figure 3.4 Relational database schema	21
Figure 3.5 Vector database schema	22
Figure 3.6 Deployed containers	25
Figure 3.7 Orchestrator	26
Figure 3.8 Filter-then-search	28
Figure 3.9 Search-then-filter	29
Figure 3.5.5.1 Bootstrap procedure for estimating a 95% confidence interval for mean latency	35
Figure 3.5.5.2 Bootstrap procedure for estimating a 95% confidence interval for the mean latency difference between two configurations	36
Figure 4.1 Mean orchestrator latency by query and configuration	39
Figure 4.2 Matched-query orchestrator latency differences	43
Figure 4.3 Strategy-group retrieval quality	44

List of Tables

Table 3.1 Benchmark query configuration breakdown	32
Table 3.5.5.1 Settings changed and held constant in configuration comparisons	34
Table 4.1 Overall outcomes	38
Table 4.2 Matched relational comparisons	41
Table 4.3 Matched execution order comparisons	42
Table 4.4 Result counts by execution order	45
Table 5.1 Assessment of project objectives	47
Table A.1 List of Endpoints	58
Table A.2 Request options	59
Table B.1 Human language interpretation of sample queries	61

1. Introduction

This report begins by establishing the way this project aims to contribute to addressing the challenges involved in the enterprise retrieval systems. It reviews the related work and then moves on to explain the methodology adopted in the design of the orchestrator, followed by the results recorded in the use of the orchestrator against a gold standard benchmark, and lastly, the evaluation and discussion of those results.

1.1 Problem Statement

The advent of large language models (LLMs) [1] has opened up numerous applications where a conversation can be had with an LLM using the knowledge encoded in the parameters of the model during pre-training. While fascinating, this has a limitation when it comes to their use within enterprises. Enterprise data is often gated and hence is not included in the pre-training of the LLMs. This can lead to hallucinations where the LLM confidently responds to queries with false information where it does not have innate knowledge. Retrieval augmented generation (RAG) [2], while based on techniques long known in the domains of search and recommendation systems, has been found to be a reliable way to ground the responses of LLMs in relevant and recent information to reduce hallucinations [3].

RAG is extensively used to provide gated enterprise knowledge to LLMs to ground their responses. Depending on the shape of information, the enterprise knowledge is often distributed across several knowledge stores such as vector stores, document stores, relational tables and graph databases. As this knowledge grows, the knowledge retrieval systems start taking the form of a distributed system where these knowledge stores need to be either partitioned or replicated or both depending on whether consistency or availability is targeted [4].

Also, in enterprise environments, deterministic orchestration is often preferred because retrieval decisions are predictable and auditable, allowing system designers to understand why particular data sources were queried for a given request. However, the performance implications of adopting deterministic orchestration strategies in distributed hybrid retrieval systems remain insufficiently understood.

Little work evaluates how different deterministic orchestration strategies influence end-to-end retrieval latency and retrieval quality when semantic and structured retrieval must be coordinated across distributed knowledge stores. Consequently, practitioners have limited evidence regarding which orchestration strategies provide

the best trade-off between latency and retrieval quality under different sharding configurations.

Lastly, in any information retrieval system, it is important to maintain a balance between the speed and accuracy of the response served to the user to maintain a higher degree of user delight [5]. This project addresses these concerns through its specific contribution.

1.2 Contribution

This project contributes the design and implementation of a deterministic query orchestrator and an experimental comparison of routing and execution strategies within a common sharded retrieval system. It evaluates how these strategies affect retrieval latency and effectiveness across defined workloads and sharding configurations. Here, deterministic strategies follow explicitly defined routing and coordination rules for a given input and system state.

In the information-retrieval literature, hybrid retrieval [6] commonly refers to combining sparse lexical and dense semantic retrieval signals into a unified ranking. The system developed in this project does not perform score or rank fusion. Instead, it is a query orchestrator for a hybrid retrieval system (HRS) supporting structured relational, dense semantic, and sparse lexical retrieval over aligned distributed data through unified orchestration. Here, aligned data refers to corresponding information represented across the different retrieval stores.

1.3 Aim and Objectives

The aim of this project is to design and implement a deterministic query orchestrator for a distributed HRS, and experimentally evaluate how routing and execution strategies affect retrieval latency and effectiveness across defined workloads and sharding configurations.

To achieve this aim, the project has the following objectives:

1. Design and implement a query orchestration framework exposed through a stateless hypertext transfer protocol (HTTP) application programming interface (API) supporting structured relational, dense semantic, and sparse lexical retrieval over aligned distributed data.
2. Implement and compare deterministic shard-routing strategies, including broadcast and selective shard querying, and execution strategies, including filter-then-search

(FTS) and search-then-filter (STF), for applicable query types.

3. Establish a reproducible evaluation benchmark comprising development and held-out test queries, with documented ground-truth results and relevance judgements appropriate to each query type.

4. Evaluate the strategies under controlled workloads and sharding configurations using mean, P95 and P99 retrieval latency [7], precision, recall and F1-score [5], and NDCG@k for ranked results [8]. For known-item queries, measure success as the proportion of queries for which the designated target appears within the top k results.

5. Analyse the trade-offs between latency and retrieval effectiveness, identifying the workload and configuration conditions under which each strategy offers advantages, and explaining the limitations of the findings.

The sections following literary review justify the methods selected to address these objectives, describe their implementation, and evaluate the extent to which the objectives were achieved.

2. Literature Review

This project lies at the intersection of RAG, natural language to SQL (NL2SQL) [9], federated search, vector search, relational databases, and distributed systems. Consequently, the literature review draws upon contributions from each of these domains to motivate the proposed architecture and identify the research gap.

2.1 RAG

RAG has received considerable attention from the research community. Xu et al. [10] propose a distributed RAG (DRAG) system using a peer-to-peer (P2P) network of knowledge stores and local LLMs. Their healthcare-focused approach introduces a novel algorithm that retrieves information locally before querying neighbouring peers. While the system is distributed, it relies solely on vector embeddings [3] as grounding information for LLMs and therefore does not address an HRS, where structured data must also be queried. Similarly, Guerraoui et al. [11] acknowledge the distributed nature of modern retrieval systems and propose RAGROUTE, a neural network classifier that predicts which knowledge store should be queried. Although this reduces unnecessary communication, the routing model is probabilistic and requires retraining whenever the underlying knowledge stores are updated. Recognising that enterprise retrieval extends beyond embeddings, Yu et al. [12] propose TableRAG, which embeds relational table schemas alongside textual content so that retrieved schema embeddings can guide subsequent SQL queries. While this demonstrates the value of hybrid retrieval, the paper does not discuss schema evolution common in large distributed tables are handled in practice.

2.2 NL2SQL

As structured data becomes an increasingly important component of HRS, the quality of NL2SQL translation has a direct impact on retrieval quality. Existing approaches can broadly be categorised as rule-based, neural and hybrid methods. Shalaan et al. [13] propose Guided SQL (GSQL), which combines rule-based processing, natural language processing and vector embeddings to improve SQL generation. SQLizer adopts a different approach by first constructing a query sketch before iteratively materialising it against the database schema using CoreNLP and Word2Vec [14]. More recently, Liu et al. [9] observe that although rule-based systems provide explainability, they struggle with adaptability and complex schemas, while neural approaches remain constrained by training data. Consequently, the field has shifted towards LLM-based methods and fine-tuning techniques that demonstrate stronger generalisation across complex database schemas [9]. Given this direction,

implementing a custom NL2SQL engine would contribute relatively little to the primary aim of this project, which is the design and evaluation of query orchestration rather than query translation itself.

2.3 Resource Selection

Once queries have been translated, the retrieval system must determine which resources should be searched. Federated search addresses this problem by providing a unified interface through which multiple independent collections can be searched and their results combined [15]. Garba et al. [15] identify CORI [16] and ReDDE [17] as two of the most influential resource selection techniques. Callan et al. [16] use probabilistic inference networks to estimate the likelihood that a collection contains relevant information. Here, CORI relies primarily on term frequency (TF) and inverse document frequency (IDF), limiting its ability to capture semantic similarity. Si et al. [17] improve resource selection by estimating collection sizes and emphasise that reducing communication between distributed collections becomes increasingly important as the number of knowledge stores grows. Although these approaches were developed for keyword-based search rather than an HRS, these studies establish that resource selection remains a fundamental challenge in distributed retrieval.

2.4 Vector Search

Within a selected resource, semantic retrieval identifies relevant items by representing documents and queries as vector embeddings, allowing similarity to be measured using functions such as cosine similarity or Euclidean distance rather than exact keyword matching [4]. However, efficiently searching large vector collections presents another challenge. Flat indexes provide exact retrieval but exhibit linear search complexity, which can become costly as dataset size and query volume increase [4]. Modern vector databases therefore employ approximate nearest neighbour (ANN) indexes such as inverted file index (IVF) and hierarchical navigable small world index (HNSW), which can reduce retrieval latency with index configuration affecting the trade-off between search speed and retrieval quality [4, 18]. More recent approaches, including DiskANN and SPANN, further improve scalability by storing large portions of the index on SSDs while maintaining lightweight routing structures in memory, enabling efficient retrieval over billion-scale datasets [19, 20]. These approaches substantially improve vector search performance but focus on optimising retrieval within individual vector stores rather than across distributed knowledge stores.

2.5 Relational Search

While semantic retrieval addresses unstructured information, enterprise knowledge also resides within relational databases. Relational systems typically employ B-tree indexes for range queries, hash indexes for equality comparisons, while PostgreSQL also provides GIN indexes for full-text and JSON search [4, 21]. Kleppmann and Riccomini [4] note that although indexes improve retrieval performance, they increase storage requirements and slow write operations because they must be maintained as the underlying data changes. Consequently, index selection represents a trade-off between query performance and maintenance overhead.

2.6 Distributed Systems

As enterprise knowledge continues to grow, a single machine may no longer be sufficient to store or process all data. Distributed database systems therefore introduce concerns such as sharding and replication. The degree of fragmentation directly influences query performance [22], while the choice of sharding strategy depends heavily on workload characteristics. Key-range partitioning is effective when queries exhibit locality, whereas hash-based partitioning generally provides a more balanced distribution of reads and writes [4]. Similar principles apply to vector databases, where semantically similar vectors may be clustered together to reduce the search space during retrieval [23]. Distributed systems also introduce opportunities for parallel query execution; however, Özsu and Valduriez [24] note that reducing response time through parallelism may increase overall execution time because of additional communication overhead. Consequently, computation should occur as close to the data as possible whenever feasible [24].

2.7 Query Planning

These considerations ultimately influence query planning within distributed retrieval systems. Özsu and Valduriez [25] argue that query optimisation should minimise communication costs by reducing the amount of data transferred between nodes. They recommend assigning execution costs to alternative query plans and favour strategies that perform selection as early as possible, thereby reducing intermediate results and avoiding expensive join operations wherever possible [25]. Such observations reinforce the importance of orchestration strategies that intelligently coordinate retrieval across multiple knowledge stores.

2.8 Synthesis and Research Gap

The reviewed literature demonstrates progress in semantic retrieval, distributed RAG,

resource selection, relational and vector indexing, and query planning. Together, these contributions show that retrieval performance depends on both the efficiency of individual stores and their coordination across a distributed system. Resource selection motivates querying only relevant collections, distributed systems research highlights communication overhead, and query planning emphasises reducing intermediate results through early selection. These findings motivate examining both which shards are queried and the order in which filtering and retrieval are performed.

However, the reviewed work does not establish how alternative deterministic orchestration strategies compare within a common sharded system supporting structured relational, dense semantic, and sparse lexical retrieval. This project addresses that gap by comparing broadcast and selective shard querying, alongside FTS and STF execution where applicable, over aligned distributed data. The evaluation examines the trade-offs between retrieval latency and effectiveness across defined workloads and sharding configurations. The following section describes the development methodology and experimental approach used to implement the system and its evaluation.

3. Methodology

A deterministic query orchestrator was developed to address the aim and objectives of the project. The two governing themes during the development were testability and reproducibility. These themes led to the development of the project in the small increments of data acquisition, schema design, shard construction, relational retrieval, vector retrieval, orchestration, end-to-end integration, and benchmarking. All these increments are covered in the following sub-sections.

3.1 Scope

In real world, an HRS receives natural language queries that may require information from multiple knowledge stores. A decision has to be made as to which knowledge stores should be queried, in what order they should be queried, and how the retrieved results should be combined before returning a response. In distributed environments, these decisions directly influence communication overhead, response latency and retrieval quality.

HRS often encounters complex queries. For example:

"Give us three movie options directed by an academy award-winning director that will motivate us to survive through tough times in life."

Such a query needs to be decomposed into the following sub-queries:

Query 1: *"Find movies directed by Academy Award-winning directors."*

Query 2: *"Find movies whose plot summaries describe inspirational stories."*

In real world enterprise applications, LLMs are increasingly used to generate such sub-queries from a complex user query and to synthesise a final response from the retrieved information [26]. These components, however, are not the focus of this project and are therefore kept outside the scope [Figure 3.1].

Instead, this project focuses designing the deterministic orchestration layer as an HTTP API responsible for coordinating retrieval across knowledge stores. In a production enterprise application, an LLM will decompose a natural language query into discreet queries the orchestrator can understand, and will request the orchestrator API to retrieve information from the aligned data in knowledge stores.

A user interface, authentication, data update endpoints, and cloud deployment are also kept outside the scope.

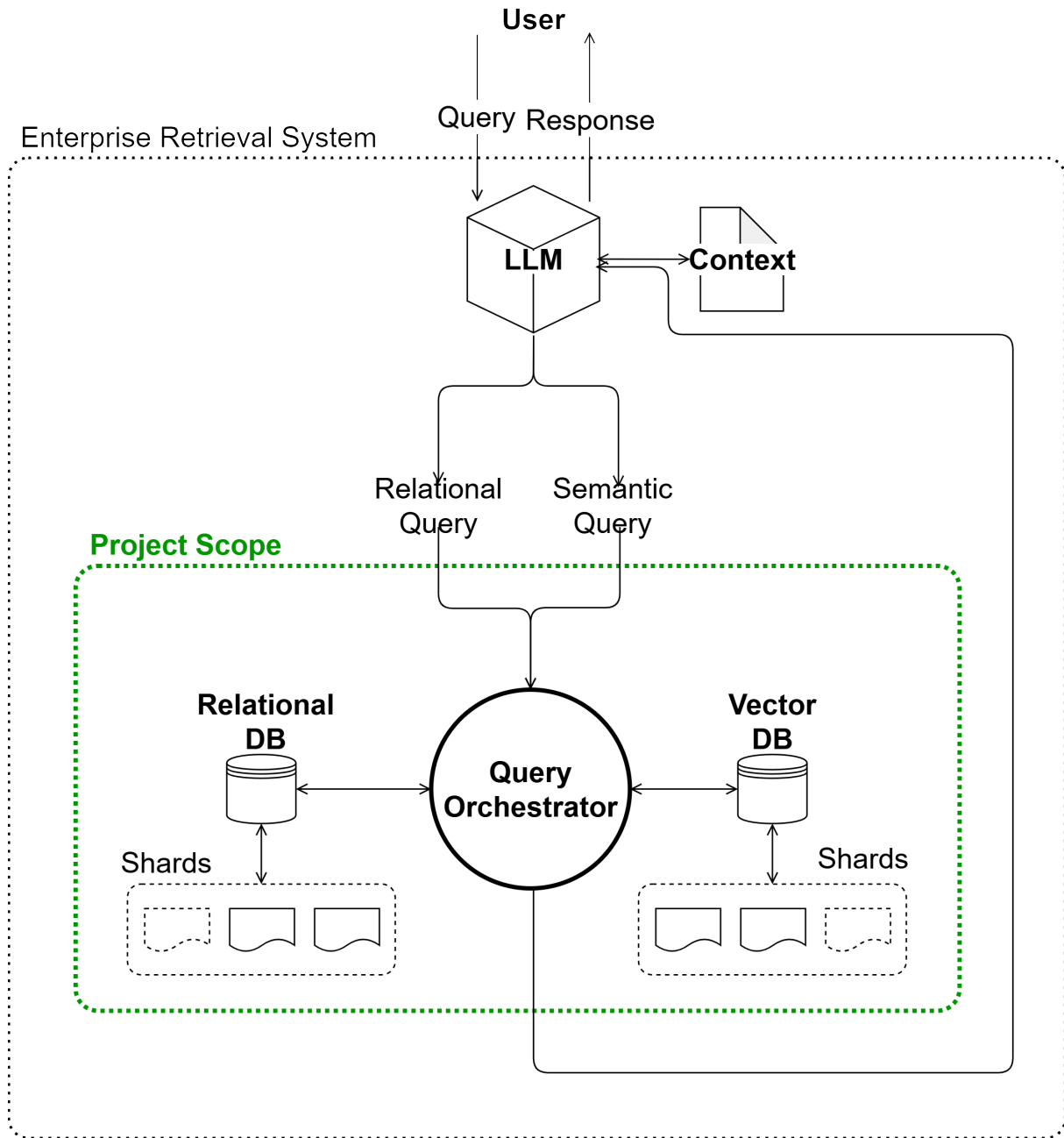


Figure 3.1 Project scope

3.2 Database

To perform evaluation of the orchestrator, it needs to be able to retrieve information from knowledge stores. Since movies contain structured data such as title, release year, director, and descriptive data such as overview, they were selected to build such knowledge stores. Among other alternatives, the movie database (TMDB) [27] was chosen for its free API access to its data, to build the knowledge stores.

PostgreSQL was selected for the relational store because its SQL support, integrity constraints, and transactions suited the structured movie metadata and relationships,

enabling precise filtering and consistent data ingestion. Qdrant [28] was selected for the vector store because it supported dense and sparse retrieval, named vectors, and metadata filtering.

3.2.1 Selection and Preparation

Using TMDB’s discovery filters, two groups were defined: U.S.-origin English-language films and Indian-origin Hindi-language films. Both applied a primary release-date range from 1 January 1990 to 1 August 2026. These specific languages and the time period were chosen for the author’s familiarity with the English and Hindi movies during the period, which was crucial for creating the known-item queries during evaluation.

Movies with a minimum 20 votes were selected in the descending order of popularity, with the exclusion of adult content. A total of 10,000 records were collected from the larger United States/English result set, while total 1,093 records from the India/Hindi group were collected. There were no duplicate movie identifiers between the groups.

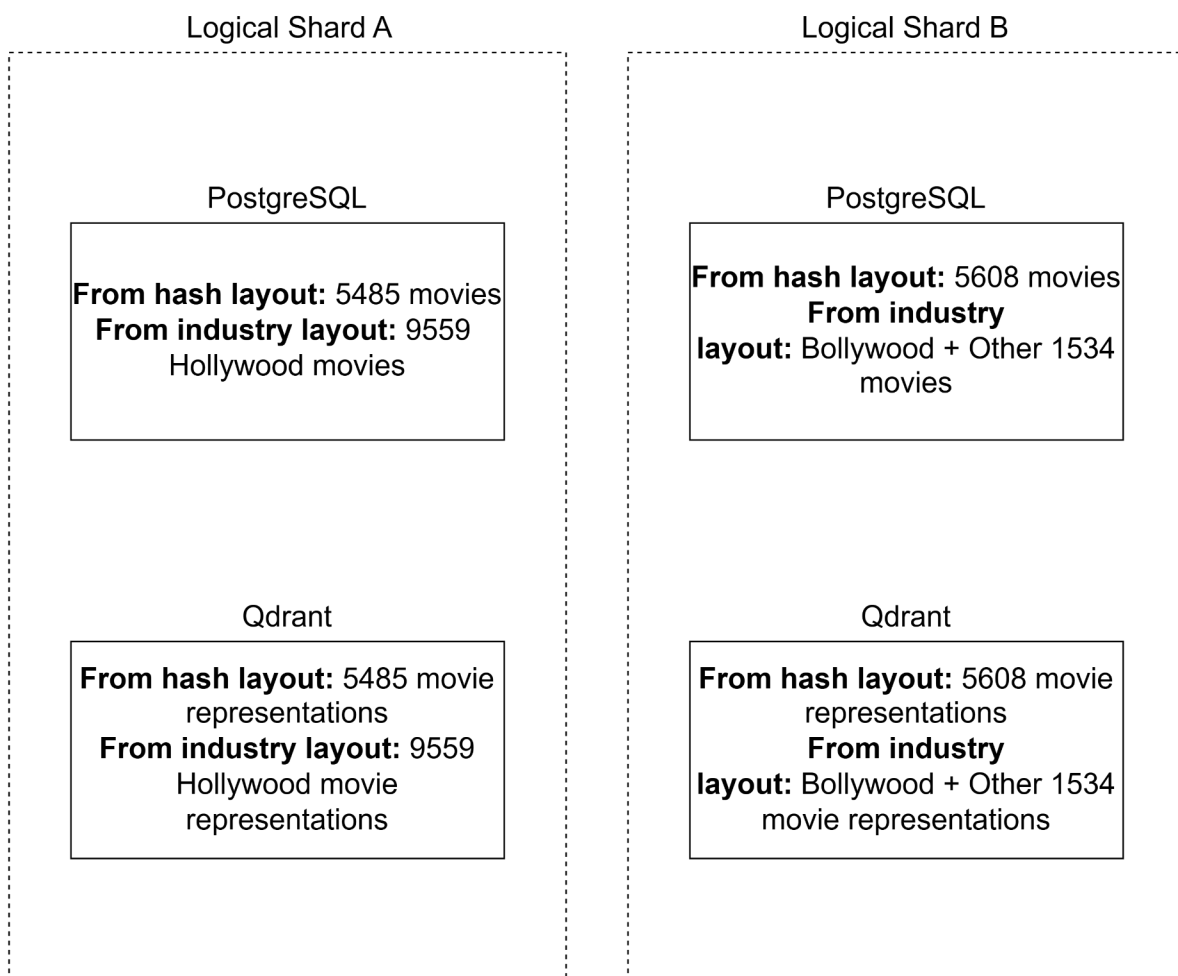


Figure 3.2 Logical shards

Records required a non-blank title and overview, resulting in a dataset of 11,093 movies.

Raw responses were cached as JSON before normalisation so that data preparation could be repeated without further API requests to TMDb. From the raw response, the canonical dataset was prepared that retained titles, overviews, release dates, genres, directors, cast, production countries, and a derived industry label. The canonical dataset served as the common input to the Python [29] loader for both the databases. SHA-256 [30] checksums [31] were created out of the canonical dataset, which were validated before loading the data so that complete parity was maintained.

3.2.2 Layout and Sharding

Figure 3.2 shows the two sharding configurations that were developed from the same canonical dataset for the experimental evaluation. The hash layout assigned even movie identifiers to shard A and odd identifiers to shard B, producing 5,485 and 5,608 movies respectively. This provided an approximately balanced distribution, although

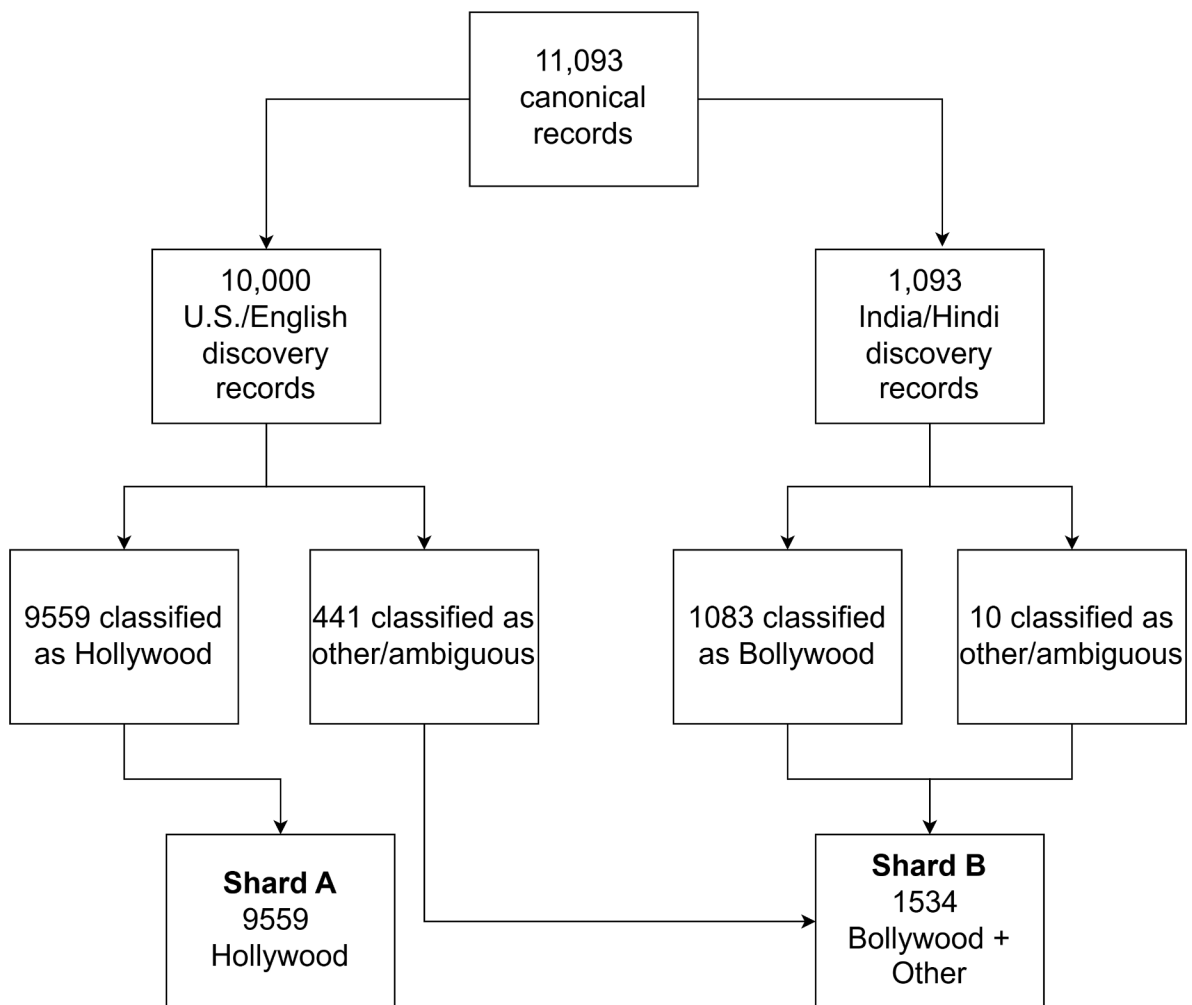


Figure 3.3 Industry classification

the assignment did not reflect the content of the movies. Figure 3.3 shows how 11,093 records were split into shards A and B.

Hashing and theme based sharding are one of the most common sharding strategies [4]. Therefore the design adopted both and also evaluated their impact during evaluation. In order to keep the experiment locally reproducible, only two shards were simulated. Simulating a larger dataset distributed across a larger number of shards is a natural future extension of this project.

3.2.3 Schema

Two complementary schemas were developed. Figure 3.4 shows the PostgreSQL tables and their relationships, while figure 3.5 shows a Qdrant point, its named vectors and its payload. The schemas represent the same movies in forms suited to exact relational conditions and ranked retrieval respectively.

In PostgreSQL, `movies` holds the attributes belonging directly to a movie, including its title, release date, overview, runtime, votes and industry. Genres, credits and countries occupy separate tables because a movie may have several of each, and each genre, person or country may relate to many movies. The `movie_genres` and `movie_countries` use composite primary keys to prevent duplicate associations. The `movie_credits` table links movies to people and retains information specific to a credit, including actor or director role, character name and cast order. Placing those attributes on the credit preserves the distinction between a person and that person's participation in a particular movie.

Qdrant stores one point per movie within each layout's collection. Each point contains the named vectors `overview_dense`, `full_document_dense` and `metadata_sparse`. Keeping these representations on one point gives them a common identity while allowing their contents to differ. The payload has duplicated fields needed for metadata filtering: `release_year`, `genre_ids`, `industry` and `country_codes`. However, PostgreSQL remains the authoritative metadata store.

The same `movie_id` connects both databases. The canonical TMDb identifier is stored as the primary key `movies.movie_id` in PostgreSQL, as the Qdrant point ID, and as `movie_id` in that point's payload. Thus, for a movie with identifier `m`, the relational record and vector point both retain `m` rather than receiving unrelated store-specific identifiers. This avoids matching by title, which may be shared by different movies or vary in spelling, and removes the need for a separate identifier-mapping table. Association-table foreign keys also use this value to connect the movie to its

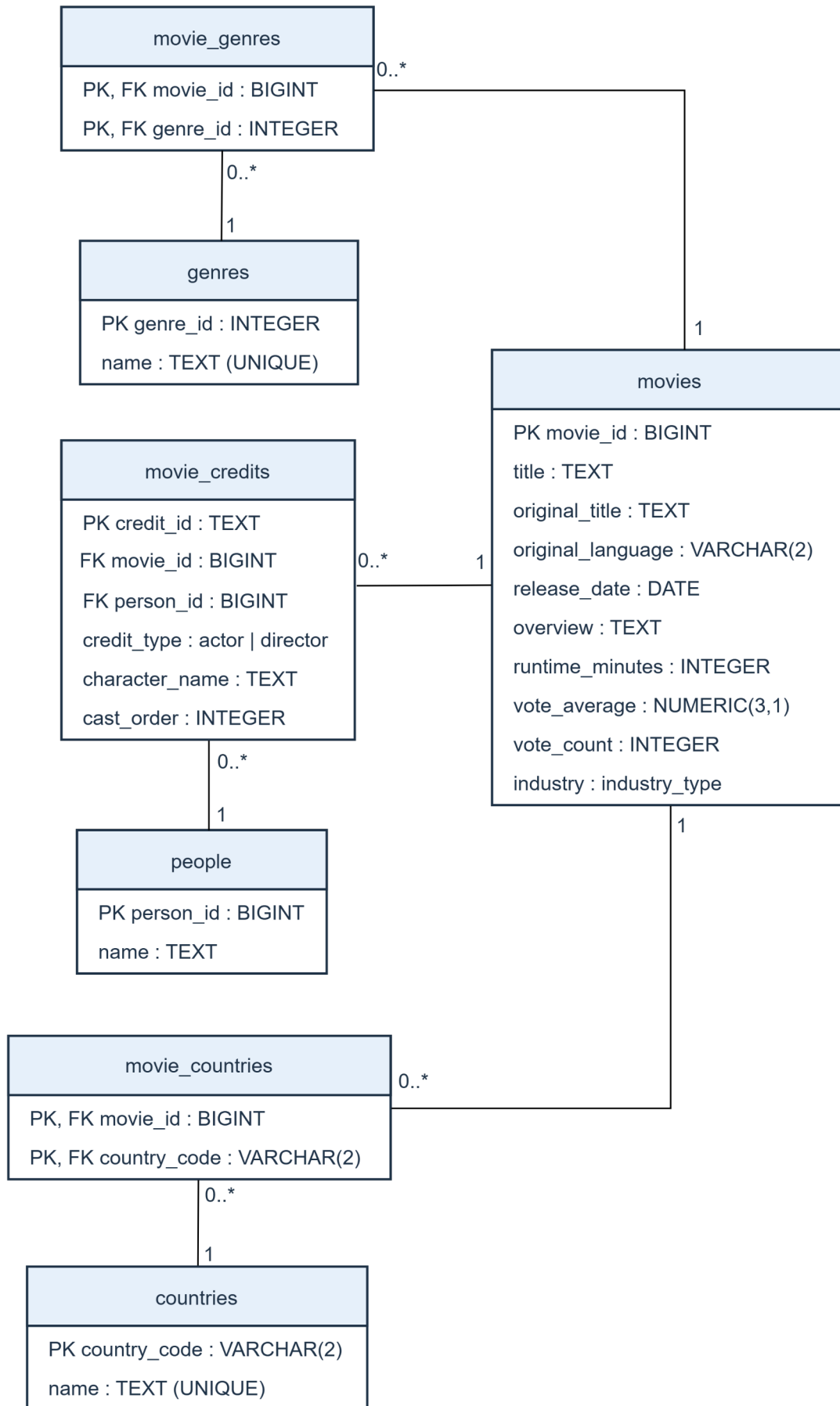


Figure 3.4 Relational database schema

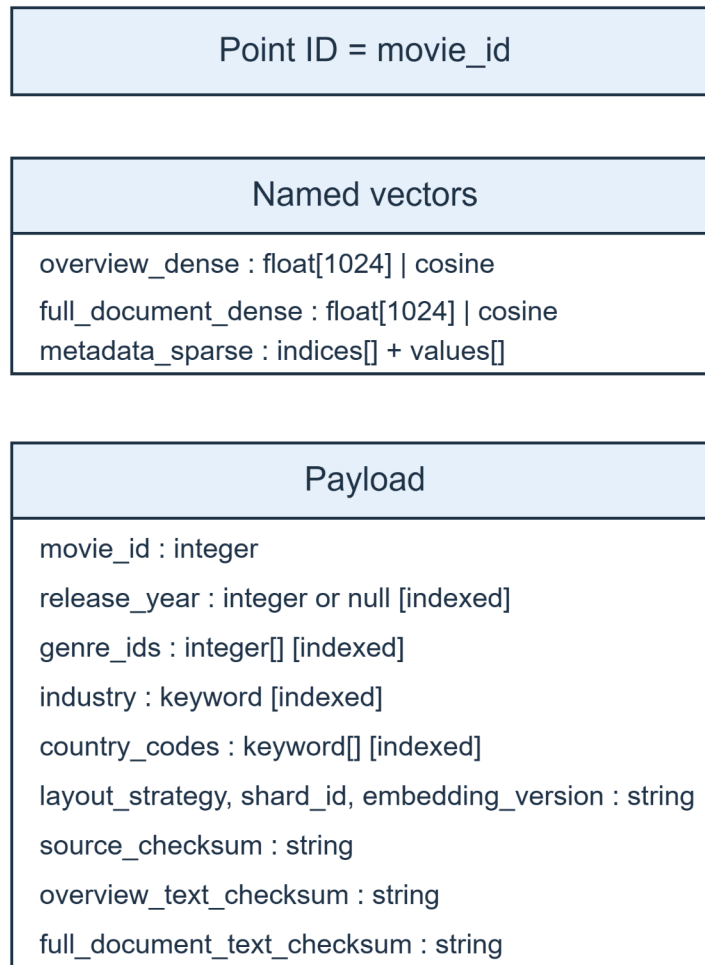


Figure 3.5 Vector database schema

relational metadata.

3.2.4 Embeddings

overview_dense embedded the overview alone. metadata_sparse encoded eight newline-separated labelled fields in this order: Title, Original title, Release year, Genres, Industry, Countries, Directors, and Cast. List entries were comma-separated; genres, countries and directors retained their canonical source order. Cast contained up to the first ten billed performers, formatted as “name as character”, or name alone when the character was absent. full_document_dense used the same metadata template followed by an “Overview:” line. White space within values was collapsed, preserving case and punctuation. Missing or blank values became “[unknown]”, and empty lists became “[none]”. Beyond the ten-person cast limit, the loader applied no character or token length truncation before passing text to the embedding libraries.

Dense embeddings were generated using text-embedding-3-large [32] with 1,024 dimensions and compared using cosine distance [33, 34]. Sparse representations were generated using Qdrant/bm25 [35] through FastEmbed [36] as term indices and

weights. The dense representations supplied semantic evidence from text, while the sparse representation retained lexical evidence from metadata. These representations were stored under the named-vector fields described in figure 5.

The 1,024-dimensional setting and selected model were not independently optimised in this project and should not be interpreted as demonstrated best choices. Dense movie embeddings were generated once, retained and loaded into both layouts, avoiding repeated external generation and ensuring that a layout comparison used the same vectors.

3.2.5 Indexes

Relational Indexes

PostgreSQL used B-tree [37] indexes on `movies.release_date`, `movies.industry`, `movies.title` and `people.name` to support date-range and equality conditions.

Relationship indexes on `movie_genres.genre_id` and `movie_countries.country_code` supported finding movies associated with a particular genre or production country. An index on `movie_credits.movie_id` supported retrieving a movie's credits, while the composite index (`person_id`, `credit_type`) supported finding movies associated with a person in a specified role, such as director.

Vector Indexes

Qdrant 1.13.6 was deployed without overrides to its default HNSW settings for dense indexing. Sparse representations used an inverted index. The dense configuration supported similarity search over the 1,024-dimensional embeddings, while the sparse index supported retrieval using term indices and weights. Qdrant's inverse document frequency modifier was enabled for sparse retrieval, so term weighting depended on the contents of each collection.

Payload Index

Qdrant also indexed `release_year` and `genre_ids` as integer fields, and `industry` and `country_codes` as keyword fields. These fields matched the benchmark's structured filters and supported restricting eligible points within the vector store during FTS.

Index definitions were consistent across layouts and were neither tuned nor evaluated as a separate design variable. They describe the configured access paths, not evidence that every request used a particular index.

3.3 Software Architecture

The orchestrator coordinates requests over the aligned data. This section describes its components, relational and vector execution paths, request validation, timing and deployment.

3.3.1 Orchestrator

A single orchestration service coordinates two PostgreSQL-Qdrant shard pairs. Two shards are used to exercise broadcast and selective querying, while keeping the experiment small enough to inspect individual results. Figure 3.6 shows this deployment topology.

Rust [38] was chosen as a language for the orchestrator mainly due its property of not having a garbage collector, which helps the deterministic orchestrator by not introducing latency related to garbage collection. Axum [39] supplied the HTTP boundary, while Tokio [40] supported concurrent asynchronous calls to the shards. Section 3.2 describes the database component.

Docker [41] Compose [42] described the service topology, network settings, and persistent database volumes. The configured database images were PostgreSQL 16 and Qdrant 1.13.6. The deployment recorded image tags, but some tags could resolve to updated images over time. Consequently, image tags alone do not establish an exact record of every installed component.

Figure 3.7 shows how the orchestrator performs request decoding, validation and coordination, database access, and enrichment. Appendix A shows the supported endpoints and Appendix B shows the examples of the requests and responses accepted and processed by the orchestrator.

3.3.2 Relational Orchestration

Relational requests specify the data layout, routing strategy, SQL statement and result limit. Selective requests also identify the shard to query. For broadcast routing, the orchestrator executes the supplied SQL concurrently on both PostgreSQL shards. For selective routing, it executes the statement only on the specified shard. The orchestrator follows the supplied routing configuration rather than deriving a route from the SQL.

Each statement executes within a read-only transaction. A transaction-local search path selects the schema corresponding to the requested layout and expires when the transaction ends.

For broadcast requests, the orchestrator concatenates the rows from shard A

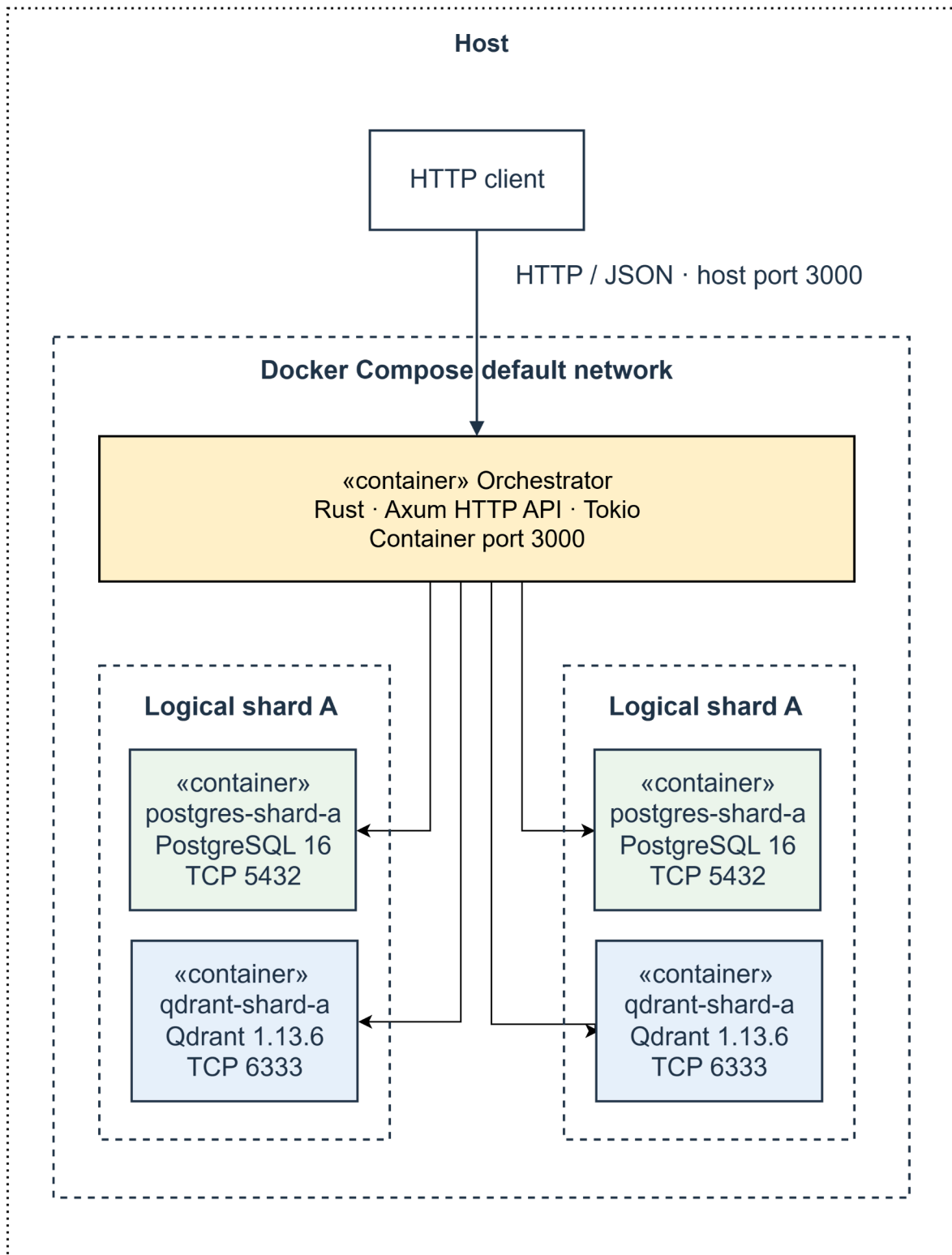


Figure 3.6 Deployed containers

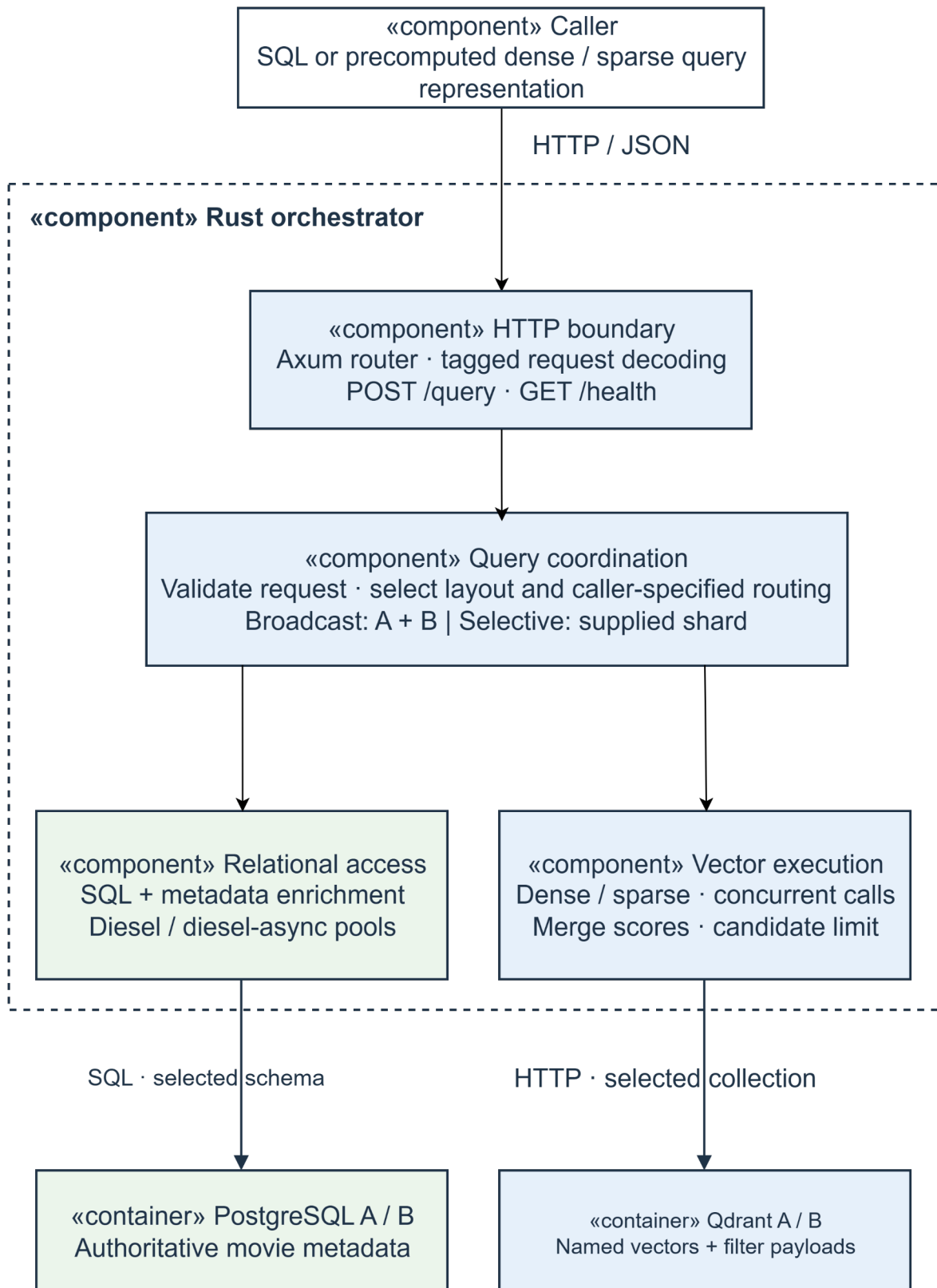


Figure 3.7 Orchestrator

followed by those from shard B, then applies the requested result limit. Ordering within each shard depends on the supplied SQL. The orchestrator does not globally sort the combined rows, remove duplicates or combine per-shard aggregates. Selective requests return rows from the selected shard, subject to the requested limit.

3.3.3 Vector Retrieval and Enrichment

Dense and sparse vector requests follow a common orchestration path. Dense requests select the vector representation to search, while sparse requests supply indices and weights. The requested layout determines the Qdrant collection, and the routing strategy determines whether the orchestrator searches both shards concurrently or only the specified shard.

For broadcast requests, the orchestrator merges the candidates returned by both Qdrant shards and sorts them by descending score. Each candidate retains its source shard and movie identifier so that its relational details can be retrieved from the corresponding PostgreSQL instance. Enrichment groups identifiers by source shard and performs the lookups concurrently. The orchestrator then reconstructs the vector ranking using the enriched records.

Filtered requests support two execution orders. In FTS, the orchestrator passes the structured conditions to Qdrant and applies them during vector retrieval. It merges the returned candidates, retains up to the requested result limit and enriches those candidates from PostgreSQL. Figure 3.8 shows this sequence.

In STF, the orchestrator requests up to 100 unfiltered candidates from each contacted Qdrant shard. It merges and ranks these candidates, retains up to 100 globally and retrieves their relational details. It then applies the saved conditions to the enriched metadata and truncates the remaining results to the requested limit. Rejected candidates are not replaced, so the response may contain fewer results than requested. Figure 3.9 shows this sequence.

The rank returned by Qdrant measures how closely a candidate matches the query. Dense retrieval uses cosine similarity between embeddings, while sparse retrieval uses a BM25-based score reflecting weighted term matches. Higher rank indicates stronger match within the selected retrieval method; they are not probabilities of relevance.

Both execution orders use the same intended filter semantics: conditions across different fields are combined with AND, alternative genres or countries within a field are combined with OR, and release-year bounds are inclusive.

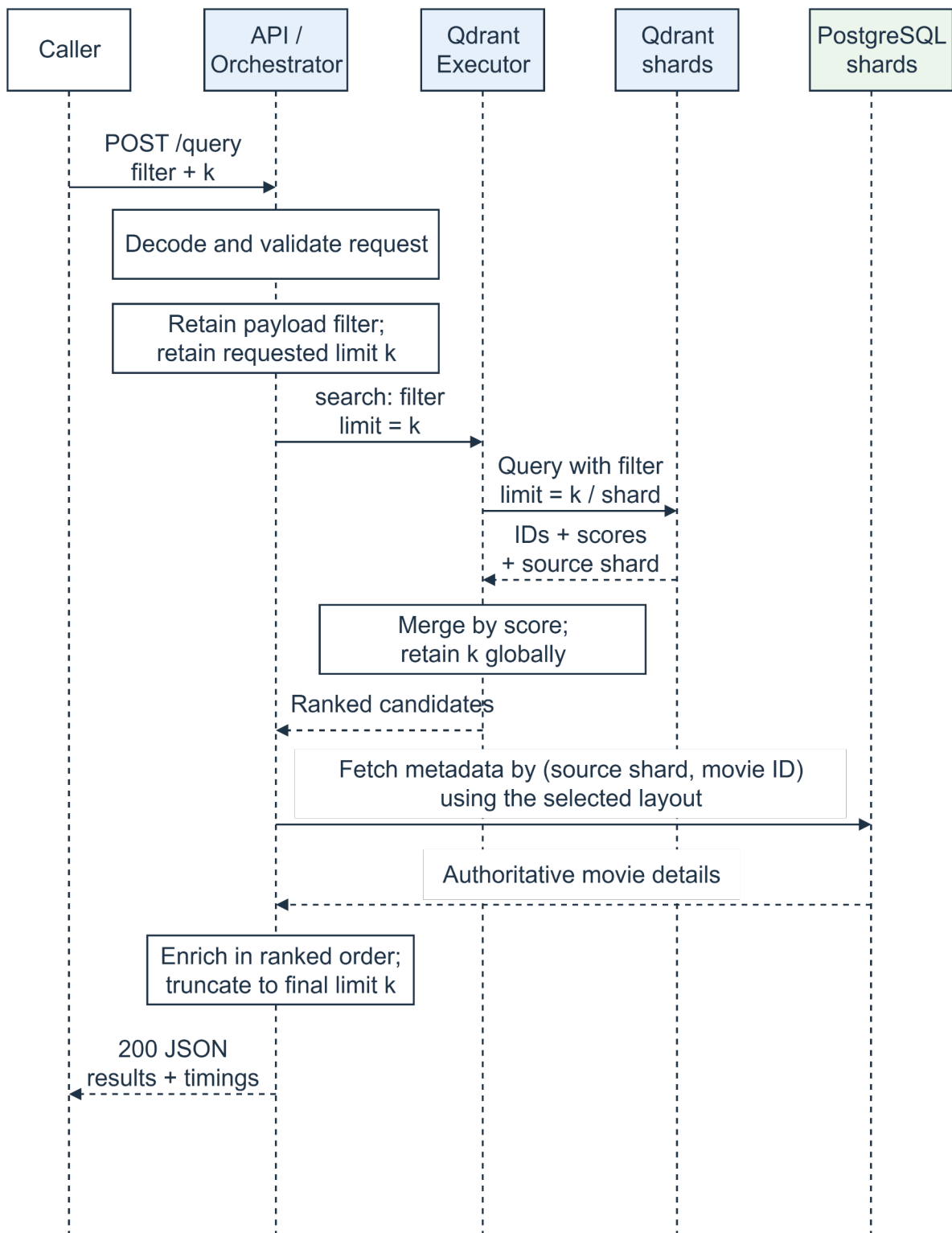


Figure 3.8 Filter-then-search

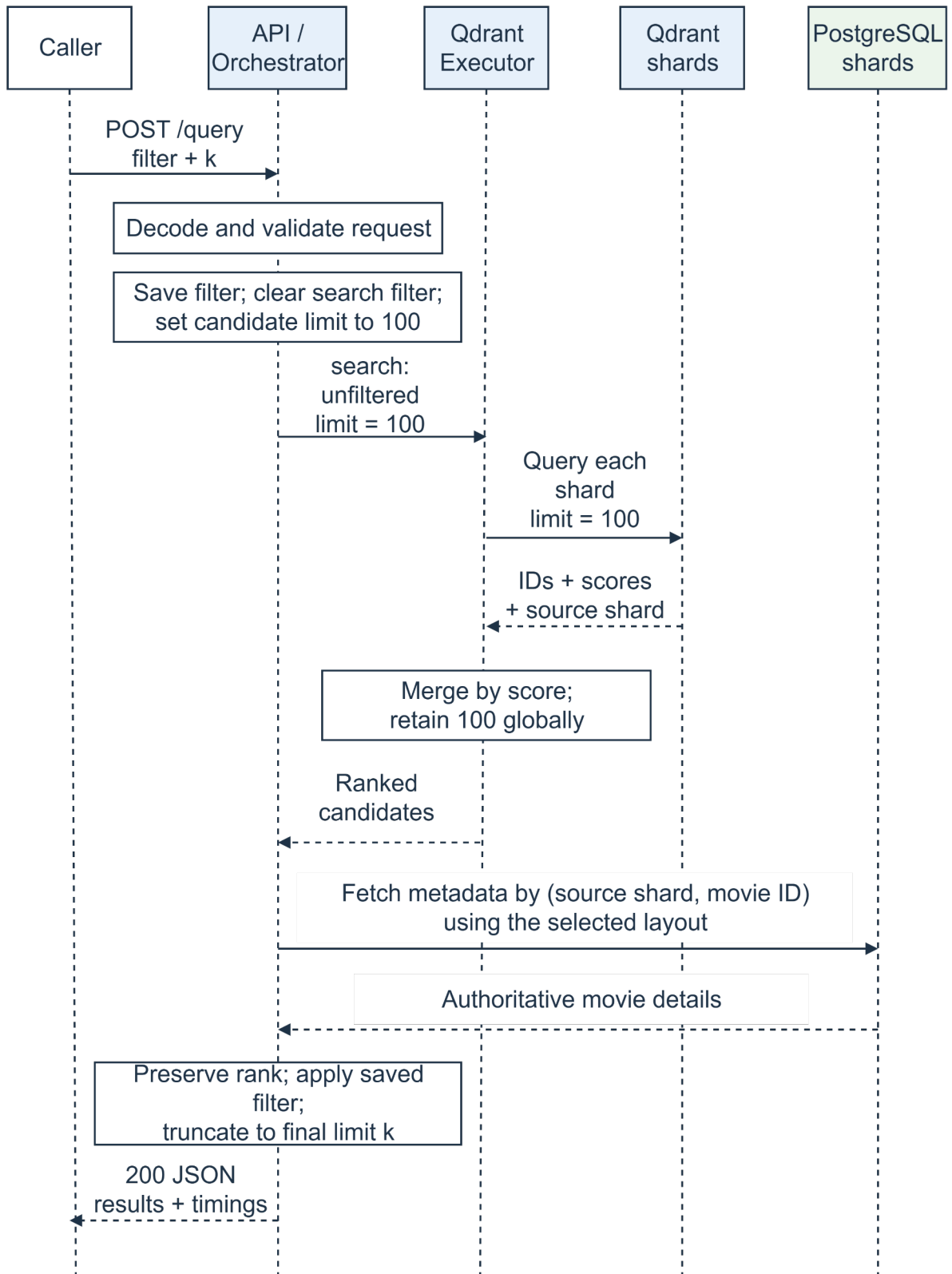


Figure 3.9 Search-then-filter

3.3.4 Validation and Timing

Validation preceded database access so that malformed requests could not become strategy-dependent database work. Invalid limits, incompatible fields, unsupported vector settings and inconsistent routing fields were rejected before execution. Table A.2 in Appendix A shows the supported options in incoming requests.

Relational responses included per-shard and total orchestration timings. Vector responses included vector retrieval time, enrichment time, and total orchestration time. Enrichment time also included post-search filtering and final truncation where applicable. Per-shard duration was not added to obtain end-to-end latency because concurrent operations could overlap.

3.4 Verification and Testing

Verification combined Python tests for data preparation and benchmark calculations with Rust tests for validation, routing, merging, enrichment, filtering and errors. Controlled mocks reproduced invalid inputs and dependency failures, and integration tests exercised the application's HTTP router against populated PostgreSQL and Qdrant stores. Appendix C details how the test requirements, and representative cases map to total 107 tests, along with the outcomes.

These results support correctness within the tested scope rather than for every possible input. Automated test totals are distinct from the repeated performance measurements described in evaluation.

3.5 Evaluation Method

The benchmark evaluated relational correctness, vector target recovery and latency across shard layouts, routing strategies and filter orders. This section describes its construction, execution and analysis procedure. Results and their interpretation are presented in chapter 4.

3.5.1 Benchmark Construction

The author manually constructed 36 logical queries: 12 relational, 12 dense-vector and 12 sparse-vector queries. As explained in section 3.2.1, the dataset's discovery filters and time period were chosen to cover Hollywood and Bollywood films familiar to the author, supporting query construction and assessment of expected answers.

Expected relational counts and movie identifiers were established by inspecting the dataset. For each vector query, the author selected a target movie first and then

wrote a query intended to retrieve it. Dense queries assessed semantic target recovery, while sparse queries assessed lexical known-item retrieval.

Six dense and six sparse logical queries included structured filters covering release years, genres, industries, countries and combinations of these conditions. Vector requests used a final result limit of ten. Appendix B illustrates request construction and responses, including examples using the benchmark's Matrix and Dark Knight query representations. These examples demonstrate some of the queries used in the benchmark. The complete workload is specified in [benchmark/queries.json](#).

Query text was encoded directly, without additional labels or appended filters. Dense representations used text-embedding-3-large at 1,024 dimensions, whereas sparse representations used FastEmbed's Qdrant/bm25 query_embed method.

Representations were precomputed and reused. All dense benchmark requests searched named vector overview_dense.

3.5.2 Configuration Coverage

A configuration combined a query with a layout, routing strategy and, for vector retrieval, execution order. Table 3.1 summarises the 109 configurations.

The benchmark did not test every possible combination. Industry predicates supported selective execution because qualifying records belonged to a known shard. The Christopher Nolan movie count query could use selective routing because all matching movies in the dataset were stored on industry shard A. In contrast, the India production-country movie query required broadcast routing because matching movies occurred on both industry shards. For unfiltered sparse queries, the target's known shard was supplied explicitly; these cases measured known-item retrieval rather than automatic shard selection or complete retrieval across the corpus.

FTS retained up to ten candidates before enrichment, while STF retained up to 100 globally before enrichment, filtering, and final truncation. These candidate limits were fixed throughout the benchmark.

3.5.3 Measurements

The benchmark assessed whether requests completed successfully, whether they returned the expected answers, and how long they took. Appendix D lists each metric, its calculation and its purpose.

Relational responses were checked against the expected counts or movie identifiers. For vector queries, Hit Rate@10 measured whether the intended movie appeared in

Query Group	Logical Queries	Hash		Industry		Filter Order	Total Configurations
		Broadcast	Selective	Broadcast	Selective		
Relational	12	12		7	6	N/A	25
Dense, unfiltered	6	6		6		FTS	12
Dense, filtered	6	12		4	8	FTS and STF	24
Sparse, unfiltered	6	6	6	6	6	FTS	24
Sparse, unfiltered	6	12		4	8	FTS and STF	24
Total	36	48	6	27	28		109

Table 3.1 Benchmark query configuration breakdown

the first ten results. NDCG@10 for dense retrieval and mean reciprocal rank for sparse retrieval measured how highly that movie was ranked. Returned-result counts showed whether filtering left fewer than the requested ten results.

Dense retrieval also included precision, recall and F1, as specified in the evaluation objectives. Each query had one designated target, so these measures mainly assessed whether that target was found. They did not assess the relevance of the other returned movies.

Client latency measured the time taken to receive and decode a response. Server timings measured the work performed inside the orchestrator, including vector retrieval and enrichment. Mean latency described average request time, while P95 and P99 described slower requests.

For each configuration, measurements were summarised across its repeated requests. Broader summaries combined requests using the same retrieval type, layout, routing strategy and filter order. Failed requests were counted separately and excluded from latency and retrieval averages. Comparisons between configurations followed the procedure in section 3.5.5.

3.5.4 Execution Procedure

The recorded benchmark used Windows 11 and CPython 3.12.14, with the orchestrator and databases deployed through Docker Compose. The run manifest

recorded checksums for the query definitions and saved representations.

Requests followed the order of query classes, queries and configurations in the benchmark specification: relational, dense and then sparse. Each configuration received one unmeasured warm-up followed by 30 measured repetitions before execution advanced to the next configuration. This specified 3,270 measured requests, executed sequentially without randomisation or interleaving. Repetitions measured variation within a configuration and were not independent benchmark runs.

Client timing began after request encoding, with a 15-second timeout. Each request's status, latency, server timings, ranking, scores and failure details were saved immediately.

3.5.5 Bootstrapping

In order to estimate the uncertainty around average latency, and the difference between the average latency from different configurations, bootstrapping was used. Both calculations used saved measurements and required no additional benchmark requests.

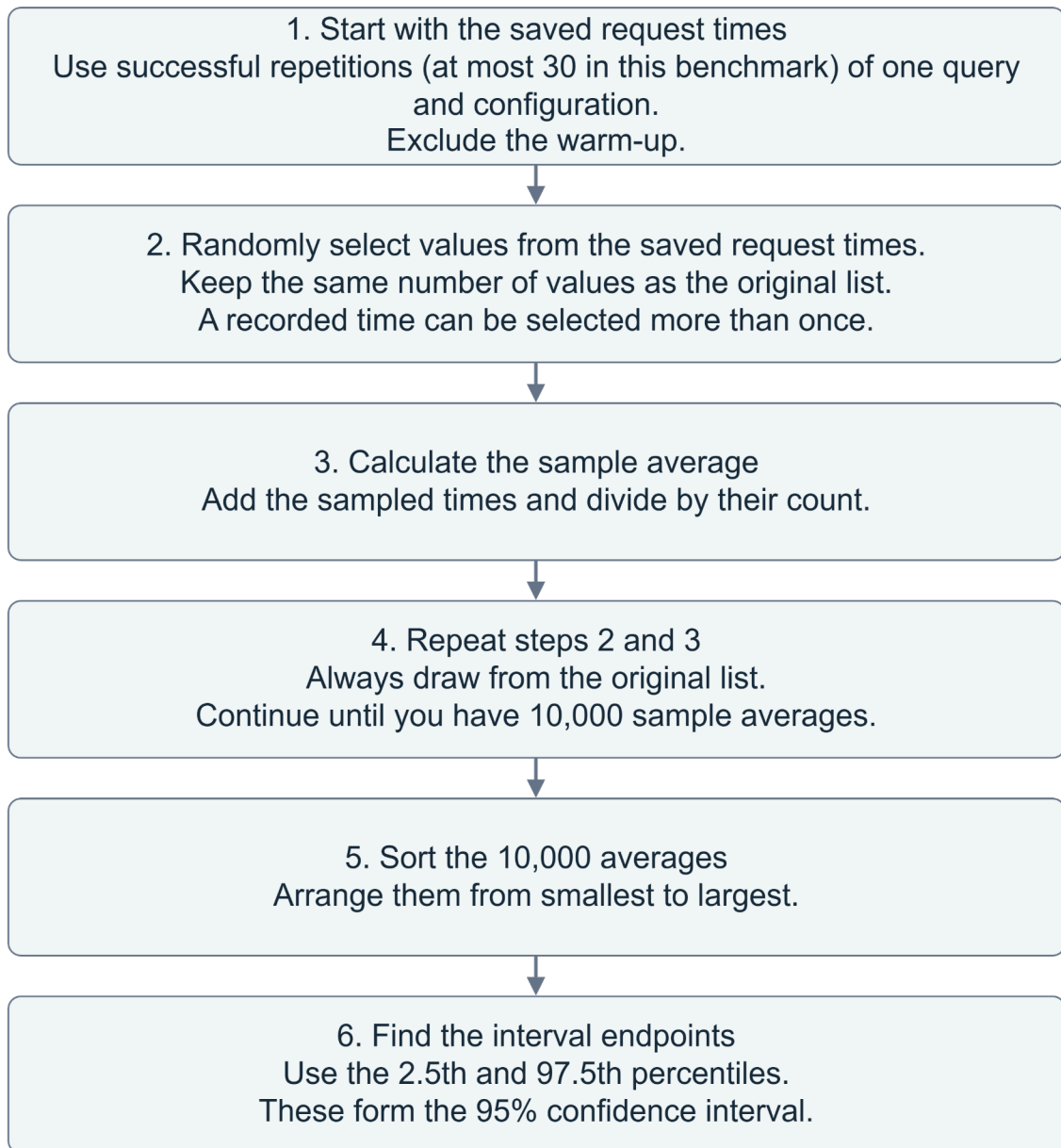
Table 3.5.5.1 shows which settings changed and which remained the same in each comparison. Only queries tested under both configurations were compared. Where multiple settings changed, the comparison measured their combined effect.

Figure 3.5.5.1 shows the bootstrap procedure for the average latency of one query and configuration. Figure 3.5.5.2 shows the procedure for comparing two configurations. For each shared query, repeated timings were averaged separately under scenario A and B. Subtracting A's average from B's produced one difference per query. These differences were then re-sampled to estimate uncertainty around the average difference.

Both procedures used 10,000 bootstrap samples and random seed 20260915 for repeatability. The 2.5th and 97.5th percentiles of the sample averages formed the 95% confidence interval. No interval was calculated with fewer than two successful timings for an individual configuration, or fewer than two shared queries for a comparison.

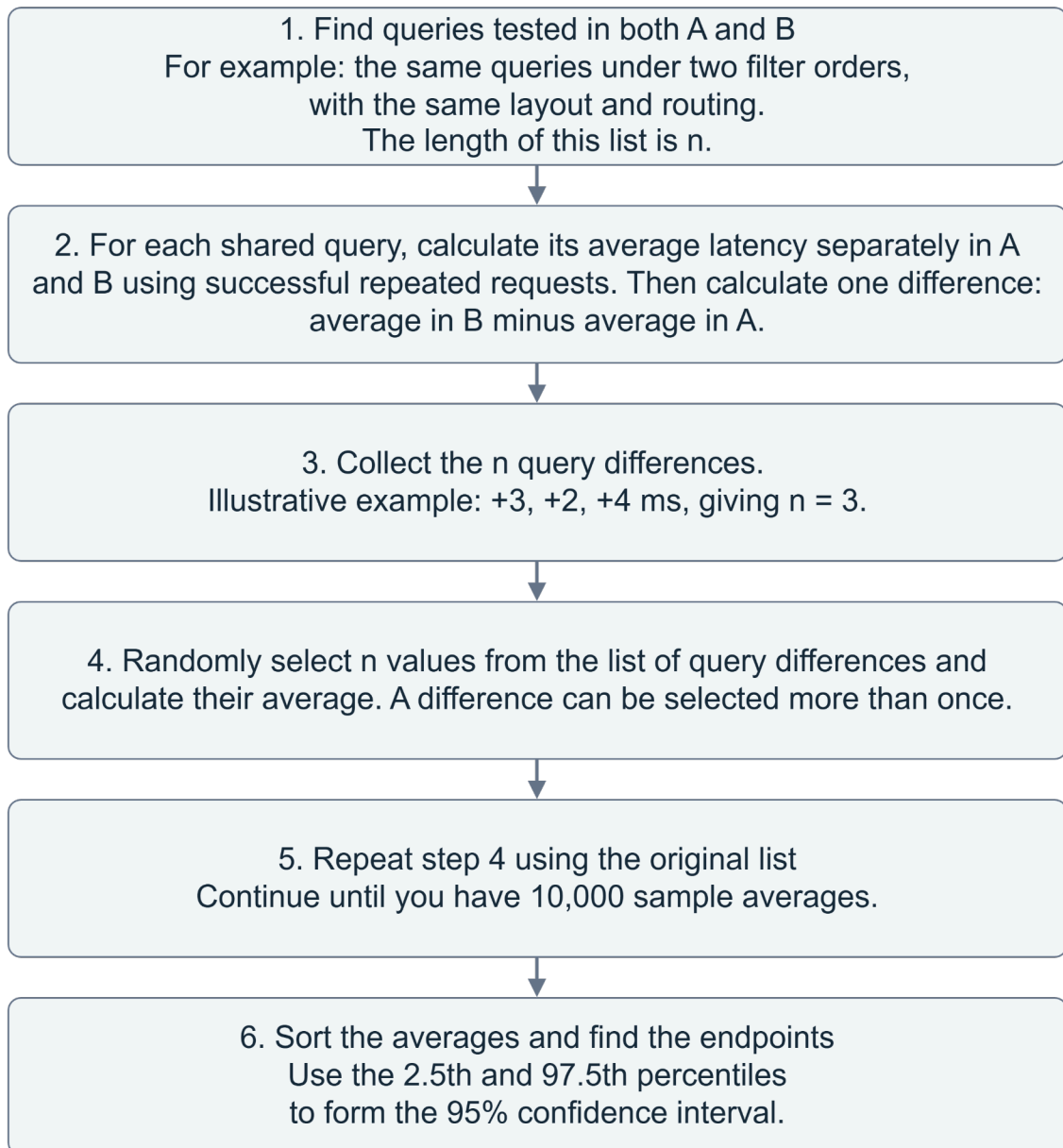
Compare	Example		Setting	
	Configuration A	Configuration B	Changed	Held Constant
Filter orders	Hash, broadcast, FTS	Hash, broadcast, STF	Filter order	Layout and routing
Routing strategies	Hash, broadcast, FTS	Hash, selective, FTS	Routing	Layout and filter order
Shard layouts	Hash, broadcast, FTS	Industry, broadcast, FTS	Layout	Routing and filter order
Combined change	Hash, broadcast, FTS	Industry, selective, FTS	Layout and routing	Filter order

Table 3.5.5.1 Settings changed and held constant in configuration comparisons



**The interval describes uncertainty in the average latency.
It is not the range containing 95% of individual request times.**

Figure 3.5.5.1 Bootstrap procedure for estimating a 95% confidence interval for mean latency



For the difference B minus A:
Interval entirely below zero: B is faster.
Entirely above zero: A is faster.
Includes zero: the comparison does not clearly favour either.

Figure 3.5.5.2 Bootstrap procedure for estimating a 95% confidence interval for the mean latency difference between two configurations

3.6 Reproducibility

Reproducibility was supported through the source code, dependency records, cached responses, canonical data, layout manifests, stored embeddings, and benchmark artifacts. Checksums linked derived files to their inputs, while saved representations avoided repeating embedding requests during evaluation.

The processing stages were deliberately separated. Data collection could be repeated from cached responses, ingestion followed the shared manifests, and benchmark analysis operated on completed request records. This made the origin of the data used by each stage traceable.

These controls supported reconstruction of the implemented system but did not remove every source of variation. Mutable container tags, host configuration, and database defaults could affect a later deployment. The recorded run environment and unresolved configuration details must therefore accompany the reported results.

3.7 Data Use and Credentials

The project used movie metadata for an academic prototype and did not collect user conversations or private enterprise documents. External API credentials were supplied through environment variables. The local deployment also contained development database credentials in its Compose configuration and was not designed as a production access-control system.

The use and presentation of the source data remain subject to TMDB's terms and attribution requirements.

4. Results and Analysis

This chapter presents the recorded benchmark outcomes and examines their implications for routing and execution order. All numerical results use the artifact set run-2026-09-16; the earlier run and diagnostic requests are not pooled with it. Comparisons use the methods defined in Section 3.6, with matched logical queries as the basis for claims about strategy differences.

4.1 Benchmark Outcomes

The benchmark completed 3,270 measured requests across 109 configurations, with 30 repetitions per configuration and no recorded request failures. All 750 relational checks matched their expected results. Table 4.1 summarises the workload and effectiveness measures.

The dense averages were Precision@10 = 0.089, Recall@10 = 0.889 and F1@10 = 0.162. With one judged relevant target per query, a successful request has

Retrieval type	Logical queries	Configurations	Measured requests	Recorded outcome
Relational	12	25	750	750 of 750 correctness checks passed
Dense semantic	12	36	1,080	Hit Rate@10 = 0.889; binary NDCG@10 = 0.827
Sparse known-item	12	48	1,440	Hit Rate@10 = 1.000; MRR@10 = 0.990

Overall outcomes. Effectiveness averages include the applicable configurations and repetitions, so queries with more configurations contribute more observations. They are descriptive totals rather than equal-weight comparisons of strategies.

Table 4.1 Overall outcomes

Precision@10 = 0.1 even when its remaining results are useful. These values therefore measure recovery of the designated targets, not the proportion of semantically relevant movies established by exhaustive assessment.

Successful execution and successful retrieval must also be distinguished. Every dense request completed, but some did not return its designated target within the first ten results. Conversely, the absence of failures in this run establishes successful execution under the measured conditions, without establishing behaviour during outages or sustained load.

Figure 8 provides an overview of server orchestration time across all configurations. Each row identifies a query, layout, routing strategy and execution order; None means that filter order is not applicable to a relational request. Bar length shows the

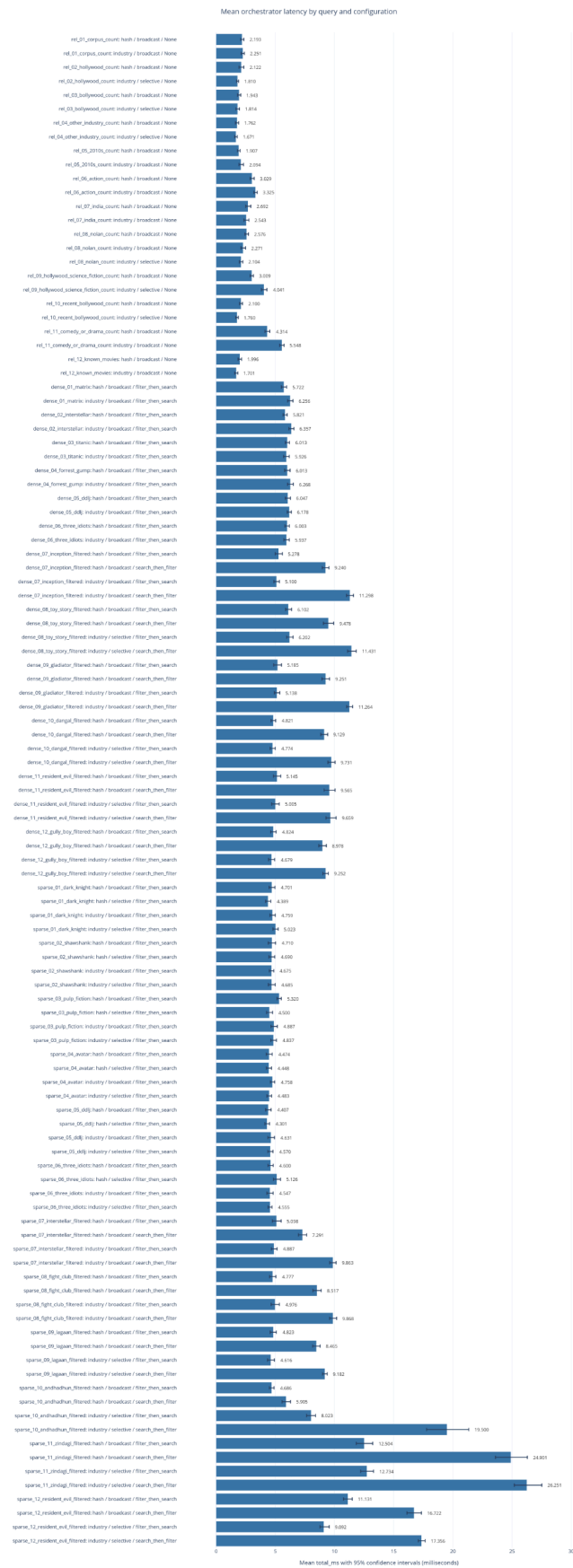


Figure 4.1 Mean orchestrator latency by query and configuration

mean total_ms, and the error bars show 95% bootstrap confidence intervals for that mean over 30 repetitions. These intervals describe uncertainty in the mean, not the range containing 95% of individual request times.

The chart shows why a single overall average would conceal useful variation. For the dense Inception query under hash-broadcast execution, mean orchestration time increased from 5.278 ms with FTS to 9.240 ms with STF. The sparse Zindagi query under industry-selective execution increased from 12.734 to 26.251 ms. These examples show that the size of the difference depends on the query as well as the strategy. The matched analysis in Section 4.4 assesses whether the pattern extends across shared queries; visual overlap between individual configuration intervals is not a substitute for that comparison

4.2 Verification Findings

Verification exposed two ways in which nominally equivalent strategies could acquire different semantics. STF initially transformed requests before validating them, allowing invalid caller settings to bypass checks. Validating the original request removed that inconsistency. Separately, PostgreSQL's industry label other differed from the canonical and Qdrant label other_or_ambiguous, causing post-search filtering to reject records that pre-filtering accepted. Restoring the canonical label during enrichment made the comparison concern execution order rather than inconsistent metadata interpretation.

Routing validity also required a data-level check rather than an assumption about the layout's name. The India-count smoke test exposed an incomplete selective route. Diagnostic measurements were excluded from the recorded results because their role was to verify the evaluation procedure, not estimate strategy performance.

The India count provides a concrete example of the difference between a plausible routing rule and a complete answer. Of 1,150 movies matching the production-country condition, only 1,086 were on industry shard B; the remaining 64 were Hollywood records on shard A. A country predicate therefore could not safely be treated as an industry predicate. Removing this selective configuration preserved the full-corpus ground truth rather than adjusting the expected answer to match an incomplete route.

These findings show why verification extended beyond checking HTTP success. The benchmark and regression cases exposed errors in validation order, metadata consistency and routing assumptions. The final run's correctness results apply to the

supported benchmark cases; they do not establish correctness for arbitrary SQL, every filter combination or every possible shard choice.

4.3 Relational Retrieval and Shard Routing

All relational configurations produced correct counts or identifier lists under the evaluator's comparison rules. Broadcast count responses contained a value from each shard, which the evaluator summed. This success should not be interpreted as evidence that the API performs arbitrary distributed aggregation: the service returns shard results and does not implement a general global reduction or ordering stage.

Table 4.2 compares relational latency on shared logical queries. Differences are B minus A; negative values favour B. Confidence intervals use the matched-query bootstrap described in Section 3.6.4.

Strategy A	Strategy B	Matched queries	Client mean difference, ms	95% confidence interval, ms
Hash, broadcast	Industry, broadcast	7	-0.014	[-2.232, 2.375]
Hash, broadcast	Industry, selective	6	-0.303	[-1.186, 0.637]
Industry, broadcast	Industry, selective	1	-1.868	Not estimated: one query

Matched relational comparisons. The second row changes both layout and routing, whereas the third holds layout constant.

Table 4.2 Matched relational comparisons

Neither of the multi-query intervals excludes zero. The measurements therefore do not provide clear evidence of a consistent relational latency improvement from changing layout or from the combined change to industry-selective execution. The one direct comparison within the industry layout favoured selective execution, but one query cannot establish a workload-wide effect.

Selective execution reduces the requested shard count from two to one, but this does not imply a proportional reduction in elapsed time. Broadcast requests execute concurrently, and client latency includes work outside the database operations. The result is consistent with a small local deployment in which avoiding one shard need not dominate the complete request cost. CPU utilisation, transferred bytes and throughput were not measured, so possible resource savings remain unquantified.

4.4 Filter Order and Vector Latency

For filtered vector queries, execution order gave the clearest latency differences. Table 4.3 holds layout and routing constant and compares FTS with STF. Each mean

is calculated over the same queries in that row.

Retrieval	Layout / routing	Queries	FTS client mean, ms	STF client mean, ms	STF minus FTS, ms	95% confidence interval, ms
Dense	Hash / broadcast	6	19.581	25.378	5.798	[4.107, 7.591]
Dense	Industry / broadcast	2	22.065	25.697	3.632	[2.662, 4.602]
Dense	Industry / selective	4	22.180	25.005	2.825	[2.410, 3.336]
Sparse	Hash / broadcast	6	21.252	26.330	5.078	[1.426, 9.158]
Sparse	Industry / broadcast	2	23.092	24.718	1.626	[-0.611, 3.862]
Sparse	Industry / selective	4	22.618	33.148	10.530	[7.192, 13.868]

Matched execution-order comparisons. Intervals are pointwise, use logical queries rather than repeated requests as samples, and should be interpreted cautiously for groups of only two queries.

Table 4.3 Matched execution order comparisons

STF had a higher client mean in every row. Five of the six intervals excluded zero; the sparse industry-broadcast comparison remained inconclusive at the client boundary. On hash-broadcast queries, FTS reduced the client mean relative to STF by approximately 22.8% for dense retrieval and 19.3% for sparse retrieval. These reductions describe the selected filtered workload, not all vector requests.

The server timings help explain the observed cost. For dense hash-broadcast queries, STF added 4.047 ms of total orchestration time, of which 2.945 ms was additional enrichment time. For the corresponding sparse comparison, the increases were 4.797 ms and 4.634 ms. Under sparse industry-selective execution, additional enrichment time was 9.233 ms of a 9.456 ms total increase. Here, enrichment includes the subsequent filtering and truncation steps.

Figure 9 presents the server-side counterparts of the client comparisons, together with the other available strategy pairs. Rows that hold layout and routing constant while changing FTS to STF lie to the right of zero, supporting the interpretation that later filtering adds work inside the service. Several routing comparisons lie close to zero with intervals crossing it, providing less decisive evidence of a benefit. The server interval for sparse industry-broadcast STF minus FTS excludes zero even though the corresponding client interval in Table 4.3 does not; the two measurements cover different parts of the request. Rows changing both layout and routing, or both layout and filter order, describe combined changes and cannot isolate either factor.

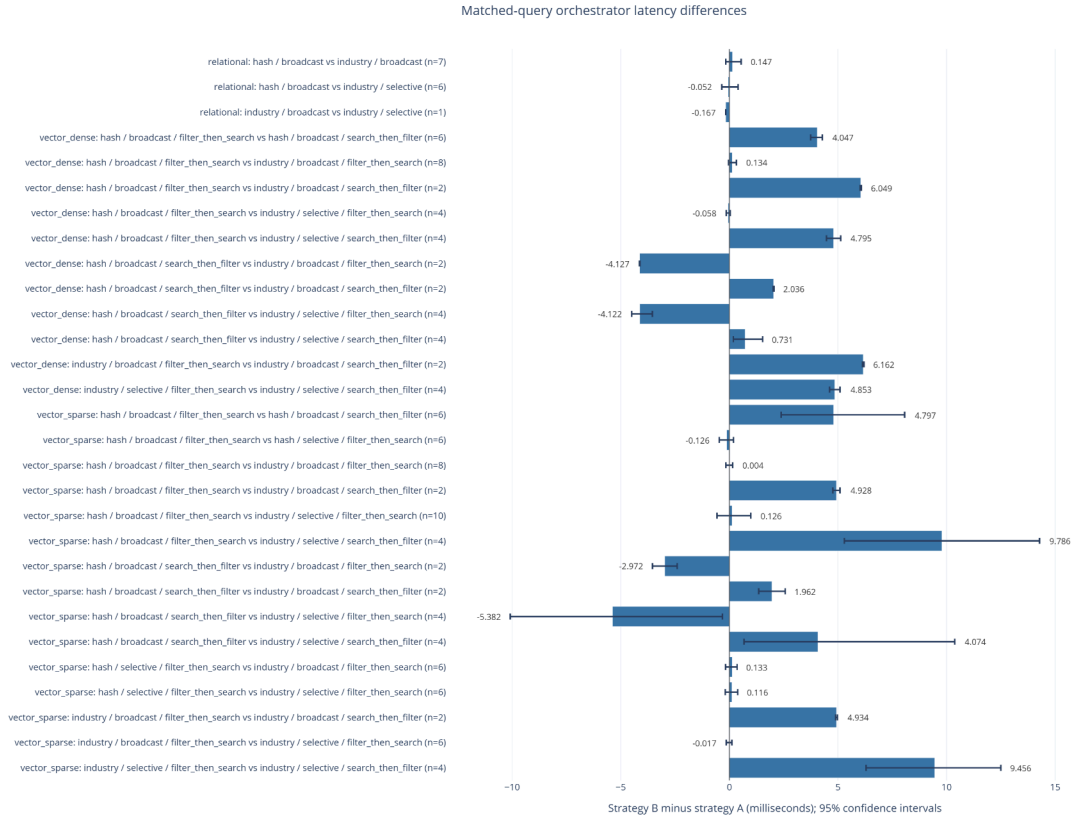


Figure 4.2 Matched-query orchestrator latency differences

This pattern is consistent with the implementation: FTS retains up to ten candidates before enrichment, whereas STF retains up to 100 globally and enriches them before applying the structured conditions. The measurements support the interpretation that processing the larger intermediate set accounts for much of the additional cost. They do not isolate the cost of each lookup, serialisation operation or filter predicate.

The fixed pool of 100 is part of this comparison. Changing it could change both cost and target recovery, and its optimum was not investigated. The results support early filtering for the tested implementation and workload without establishing that every form of post-filtered retrieval is slower.

4.5 Retrieval Effectiveness and Result Completeness

All six filtered dense queries retrieved their designated target at rank one under both execution orders in every applicable configuration. All six filtered sparse queries also placed their designated targets first. Thus, the additional STF latency was not accompanied by an improvement in the measured target-based quality of these matched cases.

The overall dense differences between strategy groups require a different interpretation. Hash-broadcast FTS had Hit Rate@10 = 0.833, whereas hash-

broadcast STF had 1.000. The first group contains all 12 dense queries and the second only the six filtered queries. Comparing those values as an execution-order improvement would confuse a change in workload with a change in strategy.

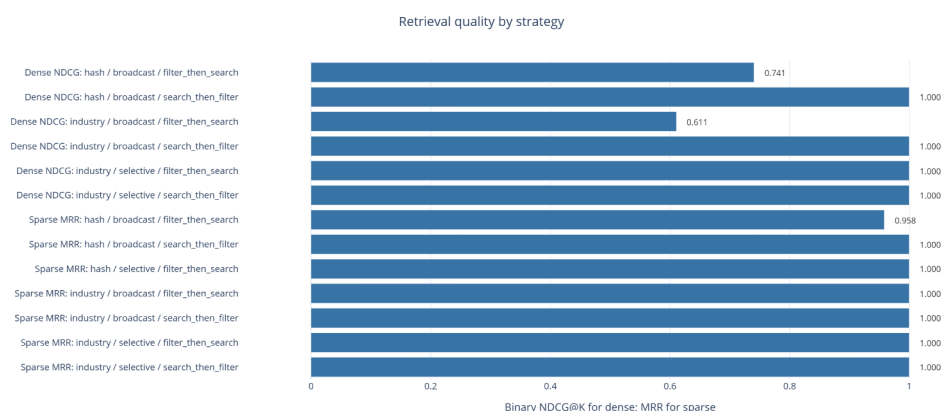


Figure 4.3 Strategy-group retrieval quality

Figure 4.1 shows dense NDCG of 0.741 for hash-broadcast FTS and 0.611 for industry-broadcast FTS, while the remaining dense groups reach 1.000. These are descriptive group averages: the lower groups include unfiltered queries whose targets were missed or ranked below first, whereas the groups scoring 1.000 contain filtered queries. The chart therefore does not establish that STF improves dense ranking. Sparse MRR is 0.958 for hash-broadcast FTS and 1.000 for the other groups, indicating a lower target position in at least one case despite successful recovery. Comparing the lengths of dense and sparse bars would not establish that one retrieval type is better, because their queries and scoring measures differ.

The unfiltered dense queries explain the lower overall effectiveness. The Matrix and Dilwale Dulhania Le Jayenge targets were absent from the top ten in both broadcast layouts. Titanic appeared at rank five, yielding binary NDCG@10 of approximately 0.387, while 3 Idiots appeared at rank three, yielding 0.500. Interstellar and Forrest Gump appeared first. These target outcomes were consistent across the two layouts in the recorded requests. They identify cases for further relevance assessment, but cannot establish whether the other retrieved movies were unsuitable because those movies were not exhaustively judged.

Sparse retrieval recovered every designated target in all configurations. The Avatar query placed its target second under hash-broadcast execution and first in its other configurations; this accounts for the overall MRR below one. Sparse scores were merged across collections without global normalisation, and the collections used their own IDF statistics. This makes score comparability a possible contributor to ranking

differences, but the benchmark contains no controlled score-normalisation experiment that would establish the cause.

Target success also conceals differences in the number of results returned. Table 4.4 illustrates this using counts verified in the raw rankings across repetitions.

Query	Layout / routing	FTS results	STF results	Designated target
Dense: Dangal	Hash / broadcast	10	6	First in both
Dense: Gully Boy	Hash / broadcast	9	4	First in both
Dense: Gully Boy	Industry / selective	9	7	First in both
Sparse: Interstellar	Hash / broadcast	10	4	First in both
Sparse: Resident Evil	Hash / broadcast	10	5	First in both

Examples of reduced result counts under STF at a requested limit of ten. Counts describe returned results rather than additional relevance judgements.

Table 4.4 Result counts by execution order

STF cannot replace candidates discarded after its global pool has been truncated. Consequently, relevant or eligible movies outside that pool cannot be returned even if fewer than ten candidates survive. These cases demonstrate a practical limitation that Hit Rate@10 alone misses. Returning more movies does not itself establish greater semantic relevance, but an application requesting a set of options may care about this difference even when its designated target remains first.

4.6 Layout, Routing and Timing Boundaries

The sparse unfiltered known-item queries allow direct routing comparisons within each layout. Across six matched queries, selective minus broadcast client latency was -0.651 ms for the hash layout, with a 95% interval of [-3.222, 1.410] ms. For the industry layout it was -1.175 ms, with an interval of [-3.654, 1.906] ms. Neither interval excludes zero. These configurations also supplied the correct target shard, so they measure execution after successful resource selection rather than the cost or accuracy of discovering that shard.

Some differences appeared at the client boundary without corresponding evidence of a server-side change. Across eight matched sparse queries using broadcast FTS, industry minus hash client latency was 3.680 ms [1.877, 5.649], while the

orchestration difference was 0.004 ms [-0.160, 0.151]. For four filtered dense queries, changing from hash-broadcast FTS to industry-selective FTS increased the client mean by 3.797 ms [1.884, 5.711], while the server difference was -0.058 ms [-0.142, 0.040]. The latter comparison changes two factors at once.

These discrepancies prevent a simple attribution of client differences to shard-search work. Client timing includes the HTTP exchange and response decoding, while server timing covers the internal orchestration path. Sequential configuration execution also permits time-varying host activity to affect the comparisons. The available timing fields do not identify which of these factors caused the discrepancy, and subtracting the timers does not isolate network latency.

Mean latency must be read alongside tail behaviour. For example, the pooled sparse industry-selective STF group had client mean, P95 and P99 of 33.148, 50.307 and 54.559 ms respectively. These are pooled percentiles over 120 requests from four queries; they are not averages of configuration percentiles. For individual configurations, 30 repetitions make nearest-rank P99 the maximum observation. These measures describe the recorded variability but do not provide a robust estimate of rare production delays.

5. Evaluation

The project aimed to implement a deterministic orchestrator and establish how routing and execution strategies affect retrieval latency and effectiveness. This chapter assesses the extent to which that aim was achieved, the validity of the evidence and the improvements needed before broader conclusions can be drawn.

5.1 Achievement of the Objectives

Table 5.1 evaluates the five objectives stated in Section 1.3 of the existing report. Implementation success is distinguished from the strength of the experimental evidence.

Objective	Assessment	Evidence and qualification
1. Implement a stateless HTTP orchestration framework for relational, dense and sparse retrieval over aligned data	Achieved within the prototype scope	Three request types share one service, with aligned identifiers and manifests, concurrent shard access and vector enrichment. The recorded run exercised all three paths.
2. Implement and compare deterministic routing and filter order	Achieved for the configured cases	Broadcast, caller-supplied selective routing and both filter orders were implemented and compared. Automatic shard selection and a fully crossed experiment were not provided.
3. Establish a reproducible benchmark with development and held-out test queries and appropriate ground truth	Partially achieved	Versioned queries, stored representations, explicit expected results, raw records and analysis artifacts support repeatability. An independently held-out test set and the full provenance of relevance assessment are not established by the available record.
4. Evaluate latency and retrieval effectiveness under controlled configurations	Achieved within the stated measurement limits	Mean, P95, P99, precision, recall, F1, binary NDCG and known-item success were recorded; MRR adds target-rank information. Single-target judgements and limited tail samples restrict interpretation.
5. Analyse conditions governing latency-quality trade-offs and explain limitations	Achieved for the selected workload	Matched comparisons identify the cost of STF, target preservation, reduced result counts and uncertain routing gains. The conclusions are bounded by the workload and single-host deployment.

Assessment against the final report's stated objectives.

Table 5.1 Assessment of project objectives

The main engineering contribution is the coordination of several retrieval paths under explicit execution rules, with consistent data alignment and traceable measurements. Its empirical contribution is narrower: a controlled comparison of those rules within one implementation. It does not establish the superiority of deterministic orchestration over a learned router because no such baseline was tested.

5.2 Interpretation in Relation to the Literature Review

The findings give concrete support, within this prototype, to the early-selection rationale discussed in Section 2.7. Applying structured conditions before enrichment reduced the intermediate set and usually reduced latency. This connection is supported both by the execution path and by the measured enrichment differences. It is more informative than observing that one strategy was faster without identifying the work it avoided.

The resource-selection arguments in Sections 2.3 and 2.6 receive more qualified support. Selective execution contacted fewer shards, but the recorded client latency differences did not consistently favour it. Broadcast operations ran concurrently, and the deployment contained only two local shards. Reducing the number of contacted resources therefore did not automatically translate into a measurable elapsed-time advantage. The experiment cannot determine whether savings would become greater with more shards, remote machines or concurrent clients.

Data placement also constrained which routes were correct. Industry partitioning enabled selective execution for some industry predicates but produced an imbalanced 9,559/1,534 split. Hash partitioning balanced record counts more closely but supplied little domain information for routing. The India-count defect illustrates why a partitioning rule must be considered together with predicate semantics. A deterministic rule is inspectable, but inspectability alone does not make it complete.

The results do not provide an absolute comparison with published retrieval latency or accuracy figures. Such a comparison would require compatible data, query tasks, hardware, timing boundaries and relevance judgements. The common implementation supplies an internal basis for comparing strategies, while the literature supplies the rationale for those comparisons.

5.3 Validity and Generalisability

5.3.1 Workload and Relevance Assessment

The 36 logical queries are a small, deliberately constructed workload over movies. Repeating them improves the description of timing variability; it does not increase the diversity of retrieval tasks. The dense judgements name one target per query, while sparse queries are known-item searches. Their scores cannot support a conclusion that sparse retrieval is generally more effective than dense retrieval because the tasks and representations differ.

There was a separate pilot, but the development history also records smoke testing and corrections using the final query definitions. A versioned query set is therefore insufficient evidence of a completely independent held-out evaluation. Without documented query authorship, assessment procedure, overlap and the point at which development decisions were frozen, the results should be treated as a repeatable benchmark rather than proof of generalisation to unseen queries. This is the principal shortfall against Objective 3.

The popularity-limited corpus and its two country/language discovery groups further restrict representativeness. Conclusions cannot automatically be transferred to other languages, enterprise documents, larger collections or more complex relational workloads.

5.3.2 Experimental Control and Statistical Interpretation

Configuration order was fixed rather than randomised or interleaved. One warm-up request may not have stabilised every cache, and requests within a configuration may share temporal effects. Matched logical-query comparisons reduce workload mismatch, but do not remove order effects or dependence between timings.

Bootstrap intervals quantify variability among the observed matched queries under the stated resampling procedure. They do not correct biased query selection or unmeasured changes in host activity. Intervals based on two queries have particularly limited information, and an interval containing zero is not proof of equivalent performance. The reported intervals are pointwise across several comparisons; no family-wide error control is claimed.

The workload was not a complete factorial design. Some comparisons change layout and routing together, and direct within-layout routing evidence is limited for relational queries and absent for the dense selective cases. Consequently, the benchmark cannot cleanly estimate every independent factor or interaction.

5.3.3 Deployment and Configuration Boundaries

The evaluation was limited to one host, two shards, one movie corpus, and the configured retrieval representations. The containers provided separately addressable stores but did not reproduce the network conditions or resource isolation of independent machines. The workload measured sequential requests and did not examine throughput under sustained concurrent use.

The vector result limit and STF candidate pool were fixed. Their effect was not independently varied, and the full-document dense representation was not evaluated. The single-target relevance judgements further restricted the interpretation of retrieval effectiveness.

Replication and node-failure experiments mentioned in the proposal were not included in the recorded benchmark. The implemented error handling could report failures, but this did not amount to an evaluation of resilience. These boundaries define the scope of the conclusions drawn in the following chapters.

The available manifest records the operating system, Python version and input hashes, but not the full processor, memory, storage, Docker allocation or immutable image digests for the measured run. These omissions limit exact reproduction of the latency environment. A later machine inspection cannot retroactively establish the original run's conditions.

Collection-dependent sparse IDF and approximate dense search also complicate interpretation across layouts. There was no global-IDF or exact-search baseline to separate orchestration effects from changes in retrieval behaviour inside the stores. Likewise, the choice of dense model was not evaluated against alternatives. These settings define the tested system rather than demonstrated optimal choices.

5.4 Engineering Strengths and Remaining Limitations

Shared manifests, stable identifiers and saved representations provided a traceable basis for data alignment. Separate validation, retrieval and enrichment components made regressions easier to locate. Returning an error when enrichment could not find an expected relational record also exposed misalignment instead of silently returning incomplete metadata.

However, alignment was maintained by the preparation workflow rather than a distributed transaction. PostgreSQL replacement and Qdrant collection population have different recovery properties, and a partially completed update could leave stores inconsistent. The prototype does not demonstrate continuous synchronisation or a recovery protocol for concurrent data changes.

The relational path also has a limited merge contract. It concatenates shard results and applies a limit without globally sorting or reducing arbitrary SQL results. This can favour shard A when the limit is reached and constrains the queries for which the returned result has the intended global meaning. Read-only execution limits writes, but caller-supplied SQL and a selected search path do not constitute a production access-control boundary.

Operationally, the health endpoint establishes that the process responds, rather than that every store is ready. Per-operation timeouts are not a single request-wide deadline. Error-response tests and the failure-free benchmark provide useful functional evidence, but do not substitute for fault injection, readiness checks, load testing or recovery measurement. These limitations delimit a research prototype rather than invalidate its measured strategy comparison.

5.5 Further Work

The first priority is to strengthen the benchmark's independence and relevance assessment. A new test set should be frozen after development, with documented query authorship and multiple assessed results per dense query. Recording assessor agreement and using graded judgements where justified would allow NDCG to measure more than the rank of a single target. Returning fewer than k results should also be reported alongside target success.

The next experiment should randomise or interleave configurations, repeat runs across sessions and record hardware and container settings. Adding valid broadcast counterparts to selective cases would make more routing comparisons possible without changing layout. Larger samples would improve tail-latency estimates, while concurrent load and multi-host deployments would test throughput, contention and communication effects.

A parameter study should vary filter selectivity, requested k and candidate-pool size. It should measure target recovery, result counts and enrichment cost together, and compare the two implemented dense representations. An exact dense baseline and a controlled sparse-score normalisation experiment would help distinguish orchestration effects from index and scoring effects.

Finally, automatic deterministic shard selection could be evaluated against an explicit routing oracle and, separately, a learned baseline. Fault injection and staged data replacement would address resilience and alignment during updates. These extensions follow directly from the present evidence gaps and would broaden the contribution beyond execution with caller-supplied decisions.

6. Conclusion

This project designed and implemented a deterministic query orchestrator for relational, dense semantic and sparse lexical retrieval over aligned sharded movie data. The service exposed a stateless HTTP API, coordinated two PostgreSQL-Qdrant shard pairs, supported broadcast and selective execution, and implemented two orders of filtering and vector search. A versioned benchmark and saved request records made the resulting comparisons inspectable and repeatable.

The 109 configurations produced 3,270 successful measured requests, including 750 correct relational checks. The strongest finding was that FTS reduced latency for the matched filtered workload while preserving every designated target. In the hash-broadcast configuration, its client mean was approximately 22.8% lower for dense retrieval and 19.3% lower for sparse retrieval than STF. Server timings associated much of the additional post-filtering cost with enrichment of the larger candidate set. STF also sometimes returned fewer results despite retaining the target first.

The routing findings were less decisive. Selective execution reduced the requested shard count, but matched comparisons did not establish a consistent client-latency advantage. Correctness depended on knowing that the selected shard contained all required records, and some comparisons changed both routing and data placement. These results show that an orchestration decision should be assessed through the work it performs, the completeness of its answer and the timing boundary being measured.

The contribution is limited to the tested corpus, representations and single-host environment. Single-target judgements restrict conclusions about semantic relevance, and the available evidence does not establish an independent held-out test set. Nevertheless, the implementation and analysis demonstrate how deterministic orchestration strategies can be compared within one aligned retrieval system. The next step is a broader, independently assessed workload with controlled routing comparisons, varied filtering conditions and distributed load measurements.

References

- [1] T. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 1877–1901. Accessed: Sep. 17, 2026. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [2] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 9459–9474. Accessed: Sep. 17, 2026. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html>
- [3] C. Huyen, *AI Engineering: Building Applications with Foundation Models*. Sebastopol, CA: O’Reilly Media, 2025.
- [4] M. Kleppmann and C. Riccomini, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 2nd edition. Santa Rosa, CA: O’Reilly Media, 2026.
- [5] C. D. Manning, P. Raghavan, and H. Schütze, “Evaluation in information retrieval,” in *Introduction to Information Retrieval*, Cambridge University Press, pp. 151–175. [Online]. Available: <https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-in-information-retrieval-1.html>
- [6] S. Bruch, S. Gai, and A. Ingber, “An Analysis of Fusion Functions for Hybrid Retrieval,” *ACM Trans. Inf. Syst.*, vol. 42, no. 1, p. 20:1-20:35, Aug. 2023, doi: 10.1145/3596512.
- [7] J. Dean and L. A. Barroso, “The tail at scale,” *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013, doi: 10.1145/2408776.2408794.
- [8] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, Oct. 2002, doi: 10.1145/582415.582418.
- [9] X. Liu et al., “A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going?,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 10, pp. 5735–5754, Oct. 2025, doi: 10.1109/TKDE.2025.3592032.
- [10] C. Xu, L. Gao, Y. Miao, and X. Zheng, “Distributed Retrieval-Augmented Generation.” Accessed: Jun. 01, 2026. [Online]. Available: <https://arxiv.org/html/>

- [11] R. Guerraoui, A.-M. Kermarrec, D. Petrescu, R. Pires, M. Randl, and M. de Vos, “Efficient Federated Search for Retrieval-Augmented Generation,” in Proceedings of the 5th Workshop on Machine Learning and Systems, in EuroMLSys ’25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 74–81. doi: 10.1145/3721146.3721942.
- [12] X. Yu, P. Jian, and C. Chen, “TableRAG: A Retrieval Augmented Generation Framework for Heterogeneous Document Reasoning,” in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds., Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 14063–14082. doi: 10.18653/v1/2025.emnlp-main.710.
- [13] H. S. Shalaan, T. H. A. Soliman, and A. M. AbdelAziz, “G-SQL: A Schema-Aware and Rule-Guided Approach for Robust Natural Language to SQL Translation,” IEEE Access, vol. 13, pp. 158520–158534, 2025, doi: 10.1109/ACCESS.2025.3607879.
- [14] N. Yaghmazadeh, Y. Wang, I. Dillig, and T. Dillig, “SQLizer: query synthesis from natural language,” Proc. ACM Program. Lang., vol. 1, no. OOPSLA, p. 63:1-63:26, Oct. 2017, doi: 10.1145/3133887.
- [15] A. Garba, S. Wu, and S. Khalid, “Federated search techniques: an overview of the trends and state of the art,” Knowl Inf Syst, vol. 65, no. 12, pp. 5065–5095, Dec. 2023, doi: 10.1007/s10115-023-01922-6.
- [16] J. P. Callan, Z. Lu, and W. B. Croft, “Searching distributed collections with inference networks,” in Proceedings of the 18th annual international ACM SIGIR conference on Research and development in information retrieval, in SIGIR ’95. New York, NY, USA: Association for Computing Machinery, Jul. 1995, pp. 21–28. doi: 10.1145/215206.215328.
- [17] L. Si and J. Callan, “Relevant document distribution estimation method for resource selection,” in Proceedings of the 26th annual international ACM SIGIR conference on Research and development in informaion retrieval, in SIGIR ’03. New York, NY, USA: Association for Computing Machinery, Jul. 2003, pp. 298–305. doi: 10.1145/860435.860490.
- [18] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” IEEE

Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[19] Q. Chen et al., “SPANN: highly-efficient billion-scale approximate nearest neighbor search,” in Proceedings of the 35th International Conference on Neural Information Processing Systems, in NIPS ’21. Red Hook, NY, USA: Curran Associates Inc., Dec. 2021, pp. 5199–5212.

[20] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, “DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node,” in Advances in Neural Information Processing Systems, Curran Associates, Inc., 2019. Accessed: Jun. 25, 2026. [Online]. Available: https://papers.nips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html

[21] “PostgreSQL 18.4 Documentation,” PostgreSQL Documentation. Accessed: Jun. 11, 2026. [Online]. Available: <https://www.postgresql.org/docs/18/index.html>

[22] M. T. Özsu and P. Valduriez, “Distributed and Parallel Database Design,” in Principles of Distributed Database Systems, M. T. Özsu and P. Valduriez, Eds., Cham: Springer International Publishing, 2020, pp. 33–89. doi: 10.1007/978-3-030-26253-2_2.

[23] J. J. Pan, J. Wang, and G. Li, “Survey of vector database management systems,” The VLDB Journal, vol. 33, no. 5, pp. 1591–1615, Jul. 2024, doi: 10.1007/s00778-024-00864-x.

[24] M. T. Özsu and P. Valduriez, “Parallel Database Systems,” in Principles of Distributed Database Systems, M. T. Özsu and P. Valduriez, Eds., Cham: Springer International Publishing, 2020, pp. 349–394. doi: 10.1007/978-3-030-26253-2_8.

[25] M. T. Özsu and P. Valduriez, “Distributed Query Processing,” in Principles of Distributed Database Systems, M. T. Özsu and P. Valduriez, Eds., Cham: Springer International Publishing, 2020, pp. 129–182. doi: 10.1007/978-3-030-26253-2_4.

[26] haileytap, “Agentic Retrieval Overview - Azure AI Search.” Accessed: Jun. 11, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/search/agentic-retrieval-overview>

[27] “TMDB API,” The Movie Database (TMDB). Accessed: Jul. 12, 2026. [Online]. Available: <https://developer.themoviedb.org/docs/getting-started>

- [28] "Qdrant - Vector Search Engine." Accessed: Jul. 12, 2026. [Online]. Available: <https://qdrant.tech/>
- [29] Python Software Foundation, "Python," ver. 3.12.10. Accessed: Sep. 24, 2026. [Online]. Available: <https://www.python.org/>
- [30] N. I. of S. and Technology, "Secure Hash Standard (SHS)," U.S. Department of Commerce, Federal Information Processing Standard (FIPS) 180-4, Aug. 2015. doi: 10.6028/NIST.FIPS.180-4.
- [31] R. Shirey, "Internet Security Glossary, Version 2," RFC 4949, Aug. 2007, doi: 10.17487/RFC4949. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4949>
- [32] "text-embedding-3-large Model | OpenAI API," OpenAI Developers. Accessed: Sep. 24, 2026. [Online]. Available: <https://developers.openai.com/api/docs/models/text-embedding-3-large>
- [33] "Distance Metrics - Qdrant." Accessed: Aug. 12, 2026. [Online]. Available: <https://qdrant.tech/course/essentials/day-1/distance-metrics/#cosine-similarity>
- [34] "Embeddings FAQ," OpenAI Help Center. Accessed: Aug. 12, 2026. [Online]. Available: <https://help.openai.com/en/articles/6824809-embeddings-faq>
- [35] "Qdrant/bm25 · Hugging Face." Accessed: Sep. 24, 2026. [Online]. Available: <https://huggingface.co/Qdrant/bm25>
- [36] "FastEmbed - Qdrant." Accessed: Sep. 24, 2026. [Online]. Available: <https://qdrant.tech/documentation/fastembed/>
- [37] "65.1. B-Tree Indexes," PostgreSQL Documentation. Accessed: Sep. 24, 2026. [Online]. Available: <https://www.postgresql.org/docs/18/btree.html>
- [38] "Rust Programming Language." Accessed: Sep. 25, 2026. [Online]. Available: <https://rust-lang.org/>
- [39] "axum - Rust." Accessed: Sep. 25, 2026. [Online]. Available: <https://docs.rs/axum/latest/axum/>
- [40] "Tokio - An asynchronous Rust runtime." Accessed: Sep. 25, 2026. [Online]. Available: <https://tokio.rs/>
- [41] "What is Docker?," Docker Documentation. Accessed: Sep. 25, 2026. [Online].

Available: <https://docs.docker.com/get-started/docker-overview/>

[42] "Docker Compose," Docker Documentation. Accessed: Sep. 25, 2026. [Online].
Available: <https://docs.docker.com/compose/>

Appendix A

A.1 HTTP Endpoints

The prototype exposes two HTTP endpoints. In the supplied Docker Compose deployment, the base URL is `http://localhost:3000`. Authentication of the API is out of scope.

The `/query` endpoint selects its execution path from `query_type`: `relational`, `vector_dense` or `vector_sparse`. It does not accept raw natural-language queries or a

Method	Path	Request	Successful Response	Purpose
GET	<code>/health</code>	No body	HTTP 200; plain text OK	Confirm that the API process responds. It does not probe PostgreSQL or Qdrant
POST	<code>/query</code>	JSON body with content-type: <code>application/json</code>	HTTP 200; JSON containing results and timings	Executes a relational, dense-vector or sparse-vector query

Table A.1 List of Endpoints

combined dense-and-sparse query type. A query's vector representations are prepared before submission. Table A.2 shows the available request options.

A.2 Errors

Errors use a JSON object with an error string. HTTP 400 covers invalid request values, malformed JSON and rejected SQL; JSON decoding or content-type failures may instead produce HTTP 422 or 415. Required-store unavailability produces HTTP 503, timeouts HTTP 504, and unreadable Qdrant responses HTTP 502. A required-store failure fails the request rather than returning partial results.

Field	Contract
layout	Required: hash or industry; selects the PostgreSQL schema and corresponding Qdrant collection.
routing	Required: broadcast contacts both shards concurrently; selective contacts the supplied shard.
shard	a or b, required for selective routing. Omit or use null for broadcast routing.
limit	Required integer between 1 and 100; maximum final result count.
sql	Relational requests only: nonempty read-only SQL query, executable as a subquery.
execution_order	Required for vector requests: filter_then_search or search_then_filter.
vector_name	Dense requests only: overview_dense or full_document_dense.
vector	Dense requests only: exactly 1,024 finite numeric values.
Indices, values	Sparse requests only: equal-length, nonempty arrays of unsigned 32-bit indices and finite numeric weights. The service selects metadata_sparse automatically.
filter	Optional for vector requests. Omit or use null for unfiltered retrieval; an empty object is rejected.

Table A.2 Request options

Appendix B

B.1 Sample Queries and Responses

This appendix covers all three accepted query types, both dense-vector targets, both routing modes and both filter orders. Examples of queries and their response can be found in the Github repository at [.\report\final\appendices\appendix B\request bodies](#) and [.\report\final\appendices\appendix B\responses](#) respectively. These are representative requests, not an enumeration of every combination of layout, routing and filter values. Both dense targets and sparse retrieval accept either layout, either routing mode and either execution order under the rules in Appendix A.

The Rust orchestrator, PostgreSQL, and Qdrant container must already be running for the queries to work. Use the <http://localhost:3000/query> endpoint. The collection does not load or modify the corpus.

Table B.1 shows the human language query associated with each of the queries found in folder. The human query is provided for easy interpretation of the query, it is not fed into the request body.

Interpretation

- B.1 queries both shards, each ordered by movie ID. The orchestrator concatenates A before B and returns at most five rows. It does not select the five smallest IDs globally or guarantee a contribution from both shards.
- B.2 restricts execution to industry shard A and filters SQL rows to Hollywood movies.
- B.3-B.6 reuse the saved embedding for the exact text computer hacker discovers that machines control the world from benchmark query dense_01_matrix. B.6 searches the same query embedding against full_document_dense; it does not generate a new embedding.
- B.7-B.9 reuse the sparse representation of the exact text The Dark Knight Christopher Nolan Batman Joker from benchmark query sparse_01_dark_knight. These terms are weighted retrieval features, not mandatory exact-match predicates.
- The release-year and industry restrictions in B.4, B.5, B.8 and B.9 are separate structured filters; they were not added to the text used to prepare the stored query

Example Name	Human Query
relational-broadcast	Show up to five movie IDs and titles from the hash layout, querying both shards.
relational-selective	Show up to five Hollywood movie IDs and titles from industry shard A.
dense-overview-unfiltered	Find up to five movies about a computer hacker discovering that machines control the world, using movie overviews.
dense-overview-filter-then-search	Find up to five Hollywood movies released from 1990 through 2020 about a computer hacker discovering that machines control the world, using movie overviews.
dense-overview-search-then-filter	Find up to five Hollywood movies released from 1990 through 2020 about a computer hacker discovering that machines control the world, using movie overviews.
dense-full-document	Find up to five movies about a computer hacker discovering that machines control the world, using full movie documents.
sparse-unfiltered	Find up to five movies matching the terms The Dark Knight, Christopher Nolan, Batman and Joker.
sparse-filter-then-search	Find up to five Hollywood movies released from 1990 through 2020 matching the terms The Dark Knight, Christopher Nolan, Batman and Joker.
sparse-search-then-filter	Find up to five Hollywood movies released from 1990 through 2020 matching the terms The Dark Knight, Christopher Nolan, Batman and Joker.

Table B.1 Human language interpretation of sample queries

vector. Year bounds are inclusive.

- B.4 and B.5 express the same human intent but execute differently, as do B.8 and B.9. Filter-then-search applies conditions during vector retrieval. Search-then-filter retains up to 100 candidates globally before applying conditions and can return fewer than five results without refilling the candidate pool.
- Vector examples search both hash shards. Results are ranked by vector score and enriched with PostgreSQL details. Similarity does not guarantee that every returned movie literally satisfies the narrative in the human query.

Appendix C

C.1 Test Requirements to Implementation Mapping Summary

C.1.1 Execute valid requests

Representative case: Submit valid relational, dense-vector and sparse-vector requests through the application's HTTP router.

Test file: tests/rust/api_end_to_end.rs

Test names:

1. relational_query_runs_through_http_api
2. dense_query_runs_through_http_api_and_enrichment
3. sparse_query_runs_through_http_api

Test level: API integration with real stores

Expected behavior: Return successful responses with the expected structure

Outcome: Passed

C.1.2 Follow caller-supplied routing

Representative case: Submit broadcast and selective requests, and check how broadcast results are combined under both shard layouts.

Test names and files:

1. broadcast_queries_both_shards — tests/rust/postgres_query_request.rs
2. selective_queries_the_requested_shard — tests/rust/postgres_query_request.rs
3. broadcast_combines_results_in_shard_order_for_both_layouts — tests/rust/postgres_queries.rs

Test level: Component and real-store integration

Expected behavior: Select the requested shards and combine relational results in shard order

Outcome: Passed

C.1.3 Reject invalid inputs

Representative case: Submit requests containing limits of 0 or 101, blank SQL, inconsistent routing settings, incompatible fields or malformed vectors.

Test names and files:

1. `rejects_invalid_execution_contracts` — `tests/rust/postgres_query_request.rs`
2. `rejects_fields_from_a_different_query_type` — `tests/rust/api_query_request.rs`
3. `invalid_vector_is_rejected_before_qdrant_access` — `tests/rust/qdrant_executor.rs`

Test level: Component and mock-service integration

Expected behavior: Reject invalid requests; reject malformed vectors before Qdrant access

Outcome: Passed

C.1.4 Validate original requests

Representative case: Submit search-then-filter requests with a zero result limit or an invalid filter.

Test file: `tests/rust/api.rs`

Test names:

1. `search_then_filter_validates_original_limit_before_qdrant_access`
2. `search_then_filter_validates_original_filter_before_qdrant_access`

Test level: API regression

Expected behavior: Reject invalid original settings before database access

Outcome: Passed

C.1.5 Merge and limit ranked results

Representative case: Return candidates with known scores from two simulated vector shards and request only the two highest-ranked results.

Test file: `tests/rust/qdrant_executor.rs`

Test names:

1. broadcast_merges_results_by_score_and_applies_the_limit

Test level: Mock-service integration

Expected behavior: Return the two highest-scoring candidates in descending order

Outcome: Passed; IDs 3 then 1

C.1.6 Preserve ranking during enrichment

Representative case: Supply two vector hits in reverse identifier order and retrieve their metadata from PostgreSQL.

Test file: tests/rust/api.rs

Test names:

1. enriches_vector_results_and_preserves_rank

Test level: Real PostgreSQL with mocked Qdrant

Expected behavior: Attach metadata without changing ranked order or the leading score

Outcome: Passed

C.1.7 Apply filter semantics consistently

Representative case: Search for a known movie using year boundaries, alternative genres and countries, and combined filter conditions under both execution orders and layouts.

Test file: tests/rust/api_end_to_end.rs

Test names:

1. both_orders_respect_inclusive_year_boundaries_in_both_layouts

2. both_orders_use_or_within_fields_and_and_across_fields

Test level: Real-store integration

Expected behavior: Include or exclude the known target according to year bounds,

OR within fields and AND across fields

Outcome: Passed: 56 combinations across both execution orders and layouts, with positive controls

C.1.8 Use canonical industry labels

Representative case: Search for movie 1576 using the other_or_ambiguous industry filter under both execution orders.

Test file: tests/rust/api_end_to_end.rs

Test names:

1. both_orders_use_canonical_other_industry_label

Test level: Real-store regression

Expected behavior: Recover the designated movie with its canonical industry label

Outcome: Passed under both execution orders

C.1.9 Report dependency failures

Representative case: Simulate unavailable dependencies, a stalled PostgreSQL connection and a PostgreSQL connection failure during enrichment.

Test file: tests/rust/api.rs

Test names:

1. failed_shard_does_not_return_partial_success
2. stalled_database_returns_gateway_timeout
3. maps_unavailable_qdrant_to_service_unavailable
4. maps_postgres_enrichment_failure_after_vector_search

Test level: Controlled-failure API integration

Expected behavior: Return service-unavailable or gateway-timeout responses as appropriate; failed relational requests return no partial results

Outcome: Passed

C.1.10 Enforce read-only execution

Representative case: Submit a SQL statement that attempts to delete records from a populated PostgreSQL shard.

Test file: tests/rust/postgres_queries.rs

Test names:

1. transaction_rejects_mutating_sql

Test level: Real PostgreSQL integration

Expected behavior: Reject the statement as invalid SQL

Outcome: Passed

C.1.11 Validate benchmark procedure

Representative case: Check the India-count query's routing configuration, calculate metrics from known inputs and execute benchmark requests containing warm-up and measured repetitions.

Test names and files:

1. test_india_count_does_not_use_unsafe_selective_routing — tests/python/benchmark/test_validate_queries.py
2. test_uses_record_state_to_calculate_each_query_type — tests/python/benchmark/test_benchmark_metrics.py
3. test_warmups_are_excluded_and_measured_records_are_written_immediately — tests/python/benchmark/test_run_benchmark.py

Test level: Python component and artifact validation

Expected behavior: Exclude unsafe routing, calculate expected metrics and omit warm-ups from measurements

Outcome: Passed

C.1.12 Detect missing relational records

Representative case: Return one valid vector hit and another whose identifier is absent from the selected PostgreSQL shard, then attempt enrichment under both

execution orders and layouts.

Test file: tests/rust/api_end_to_end.rs

Test names:

1. missing_relational_record_rejects_enrichment_without_partial_results

Test level: Real PostgreSQL with mocked Qdrant

Expected behavior: Return an enrichment error identifying the missing record, without partial results

Outcome: Passed under both execution orders and layouts

Appendix D

Metric	Calculation in the benchmark	Reason for selection
Request and failure counts	Count all measured requests and those whose recorded HTTP status was not 200.	Distinguish execution failures from successful requests that missed their target.
Relational correctness	Sum returned shard counts and compare with the expected count, or compare sorted returned movie identifiers with the sorted expected list. Count exact matches as passed checks.	Establish whether execution preserved the expected answer.
Client latency	Time from immediately before sending the HTTP request until response decoding completed. Request encoding and vector generation were excluded..	Measure latency experienced by the caller.
Vector retrieval time	From vector search to ranked candidates returned.	Identify the retrieval stage's contribution to latency.
Enrichment time	Time spent retrieving relational details, including subsequent filtering and final truncation where applicable.	Identify the enrichment stage's contribution to latency.
Total orchestration time	From entry into the orchestration method until construction of the response.	Distinguish service execution time from the broader client measurement.

Metric	Calculation in the benchmark	Reason for selection
Mean latency	Calculate the arithmetic mean of successful timing observations.	Summarise average execution cost.
P95 and P99 latency	Sort the n timings and select positions $[0.95n]$ and $[0.99n]$, counting from one	Describe slower requests that the mean can conceal. With 30 observations, P99 equals the maximum.
Hit Rate@10	Number of successful requests returning the target in the first ten positions, divided by the total number of successful requests.	Measure how consistently the designated target was recovered.
Precision@10 — dense	Number of designated relevant movies retrieved in the first ten positions, divided by ten, even when fewer results were returned.	Report the planned precision measure against the designated relevance set.
Recall@10	Percentage of the relevant movies returned in top 10 from total number of relevant movies in the relevance set.	Measure recovery of the specified relevance set.
F1@10	Harmonic mean of precision and recall	Report the planned combined precision–recall measure.

Metric	Calculation in the benchmark	Reason for selection
Binary NDCG@10 - dense	The benchmark uses the binary-relevance version of the logarithmic formula presented by Manning et al. [5]. Here, with one designated target, we assign $1/\log_2(r+1)$ if it appears at rank $r \leq 10$, and zero otherwise.	Reward recovery near the top of the ranking and penalises burying good results lower in the returned results.
Mean reciprocal rank — sparse	Assign $1/r$ when the target appears at rank r , or zero when absent, then average across successful requests.	Measure how highly the known item was ranked.
Returned-result count	Count the final results after filtering and truncation.	Identify requests returning fewer than the requested ten results, which target recovery alone would not reveal.

